

HAZOMAT

B39VS - Systems Project

Abdul Maajid Aga - H00431236

Muhammad Abban - H00427641

Laith Samir Mohamed - H00427630

Aisultan Alpysbay - H00407970

Moaiz Saeed - H00451062

Sufyaan Atif - H00434736

School of Engineering and Physical Sciences
Heriot-Watt University Dubai

April 2025

Abstract

This paper presents the design and evaluation of Hazmat, an autonomous mobile manipulator developed for indoor hazardous waste handling. The system combines a mecanum-wheeled base for omni-directional mobility, a 3-DOF robotic arm with a four-bar linkage for object manipulation, and a stereo vision system with LiDAR for perception. Navigation is achieved using Cartographer SLAM with encoder and IMU-based odometry. A micro-ROS architecture enables real-time communication between a Raspberry Pi 5 and an ESP32-S3, facilitating distributed control. Design iterations were guided by CAD modeling and Unity simulation. Experimental validation in controlled environments demonstrated reliable localization, navigation, and waste manipulation, highlighting the system's potential for deployment in industrial safety applications.

Contents

1	Introduction	3
1.1	Project overview	3
1.2	Aims and objectives	4
1.3	Roles and Responsibilities	5
1.4	Project Scope	7
2	Background and Theory	8
2.1	Literature review	8
2.1.1	Mobile base	8
2.1.2	Manipulator	9
2.1.3	SLAM (Simultaneous Localization and Mapping)	9
2.1.4	Single Board Computer	10
2.1.5	Microcontroller	11
2.1.6	Pros and Cons of Different Methods	12
2.2	Motor Control	14
2.2.1	Mecanum wheels	14
2.2.2	Wheel encoders	14
2.2.3	PID control	15
2.2.4	PWM control	16
2.3	Sensing	16
2.3.1	Odometry	16
2.3.2	Localisation and Mapping	17
3	Methodology and Implementation	17
3.1	Concept Development and Evaluation	18
3.2	Digital Design	19
3.3	Mechanical Design	21
3.3.1	Mobile Base	21
3.3.2	Manipulator	25
3.3.3	Assembly	27

3.4	Sensors	28
3.5	Software	30
3.5.1	SLAM	30
3.5.2	Vision	33
3.5.3	Embedded systems	34
3.5.4	Control	34
3.5.5	Audio feedback	35
3.6	Circuit schematics	35
3.7	Prototype development	37
4	Results	50
4.1	Computer simulation	50
4.2	Testing results	51
5	Management and Marketing	51
5.1	Project Management	51
5.2	Project Finance	52
5.3	Marketing	53
5.4	Risk Management	54
5.5	Health, Safety, and Environment	54
6	Conclusion and Future Work	54
	References	55
	Appendices	56
A	Code repository	56
B	Risk assessment matrix	56
C	Gantt chart	57
D	Meeting minutes	57

1 Introduction

1.1 Project overview

The project involves the design and development of an autonomous mobile manipulator capable of picking up waste from waste points and disposing of it at the base station.

The robot, designated 'Hazmat' manufactured by SLAM Industries, is equipped with a Raspberry Pi 5 as its main processor, assisted by an ESP32-S3 to offload computational tasks. This configuration reduces the processing burden on the

Pi and provides an additional safeguard against potential hardware faults due to faulty wiring.

Hazmat is fitted with mecanum wheels, enabling omni-directional movement and allowing it to smoothly traverse through tight spaces within a factory environment. The robot utilizes a stereo-depth camera (Oak-D Pro W) with an integrated IMU, motor encoders, and a 2D LiDAR (RpiLiDAR A3) to perform simultaneous localization and mapping (SLAM), enabling it to autonomously locate waste containers and navigate to disposal points.

Hazmat is also equipped with a 3-DOF manipulator designed using a four-bar linkage mechanism, and a dual-claw soft servo gripper for efficient waste pickup and placement. The servo motors were modified to output potentiometer feedback to the Pi, enabling real-time inverse kinematics calculations for the robotic arm. Power is supplied by a lithium polymer battery that drives the wheels, while a buck converter is used to step down voltage for the robot's electronic systems.

1.2 Aims and objectives

Aims:

- To design, fabricate, and demonstrate a functional, autonomous mobile manipulator capable of navigating an indoor environment, picking up hazardous waste containers, and delivering them to designated disposal areas with minimal human intervention.

Objectives:

- Design and fabricate a stable mobile robot based on mecanum wheels to allow omnidirectional movement in constrained indoor environments.
- Develop a secure container compartment to safely transport waste containers during robot motion.
- Integrate vision and ranging sensors (stereo camera, 2D LiDAR) to enable environmental perception, obstacle detection, and navigation.
- Design and build a 3-DOF robotic manipulator with a four-bar linkage for grasping and moving waste containers.
- Implement autonomous navigation using SLAM (Simultaneous Localization and Mapping) and obstacle avoidance algorithms.
- Implement object and container recognition using colour coding and computer vision methods.
- Develop audio feedback features to alert humans during path obstruction scenarios.
- Integrate micro-ROS middleware to allow reliable communication between the embedded control systems and the central processor.

- Implement a user interface and visual indicators (LEDs or screen) to display robot status and task progress.
- Develop a simple autonomous charging strategy (proposal stage) for future continuous operation.
- Simulate and test robot behaviour in Gazebo or a relevant simulation environment before real-world testing.
- Manage team roles and collaborate effectively across mechanical, electrical, and software domains to achieve system integration.

1.3 Roles and Responsibilities

SLAM Industries started as a military contractor, developing advanced autonomous systems for extreme environments. Our team created robots for reconnaissance, high-risk operations, and more, pushing the limits of robotics technology. Now, we have shifted focus, using our military-grade expertise to design robots for a wide range of industries. From hazardous waste management to autonomous transportation, we deliver reliable, high-performance solutions.

SLAM Industries is a team of six passionate individuals committed to redefining the future of autonomous systems. The company structure is organized as shown in Figure 1, and the key responsibilities and contributions are as follows:



Figure 1: Organisation hierarchy

Abdul Maajid Aga - Director of Robotic Operations

- Designed the robot base, top plate, and structural stands, ensuring organized component layout and accessibility.
- Co-developed the 3-DOF four-bar linkage arm with Laith.
- Migrated wheel control from serial to micro-ROS on the ESP32-S3.
- Helped wire and verify the full electrical system.
- Developed inverse kinematics algorithms for the arm (not used in final version).
- Created and refined early prototypes for mechanical testing.
- Optimized the robot's structure for easy assembly, maintenance, and manufacturing.
- 3D-printed and assembled mechanical components.
- Led the team, assigned tasks, enforced timelines, and maintained momentum.
- Provided morale support and ensured teammate well-being throughout the project.

Laith Mohammed - Senior Executive Mechanical Director

- Assisted in designing and prototyping multiple robot base plates using laser-cut acrylic until finalizing the design.
- Designed and prototyped several robotic arm versions using laser-cut acrylic to reach the final version.
- Sourced power converters and built cable harnesses for power distribution.
- Assembled a custom PCB using a perf board to securely integrate the ESP32 with the motor driver board.
- Developed a C++ library to control and read the speed of the mecanum wheels.
- Coordinated between electrical, mechanical, and software teams.

Aisultan Alpysbay - Electronics and Embedded Systems Lead

- Designed the electrical schematic, simplified diagram, and initial PCB (not used in final system).
- Assembled the robotic arm multiple times, including rebuilds after failures.
- Rewired the robot's electronics twice for improved reliability.
- Replaced damaged motor drivers.
- Researched and provided design inspiration for the final robot layout.
- Soldered additional wiring onto four servos for sensor feedback (one later replaced).
- Suggested vision-based alignment techniques to improve arm positioning.

Sufyaan Atif - Electronics Specialist and Marketing Support

- Designed the complete electrical schematic and simplified system power flow diagram.
- Gathered and organized all required electronic components.
- Assembled and wired the robot's electrical system.
- Tested DC motors, encoders, and sensors for functionality.
- Wrote code to read encoder pulses and calculate wheel RPM.
- Calibrated encoders for reliable measurements.

- Assembled the robotic arm and programmed servo operating limits.
- Tested arm movement with different joint angles.
- Managed the project website and marketing activities.

Abban Fahim - Senior Software Developer

- Interfaced sensors with Raspberry Pi, including LiDAR and stereo camera.
- Set up the complete ROS framework, launch scripts, and development environment on Raspberry Pi.
- Implemented inverse kinematics and PID controller for mobile base control.
- Created a simulation environment to test mobile robot behavior.
- Assisted in implementing SLAM and color detection nodes.
- Developed Aruco marker detection and localization system.
- Built the autonomous navigation stack, fusing multi-sensor data and odometry.
- Wrote the initial serial communication prototype between Pi and ESP.
- Programmed final I2C communication system for the manipulator.
- Mentored the team on Linux, version control, and ROS fundamentals.
- Typesetting the final report.

Moaiz Saeed - Software Specialist

- Assisted in designing the robot's base structure.
- Developed and tested the odometry ROS node.
- Supported the implementation of Cartographer SLAM using LiDAR for mapping and localization.
- Wrote the full URDF (Unified Robot Description Format) model for the robot.
- Helped set up the robot's transformation (TF) frames in ROS.
- Implemented a colour detection system using OpenCV.
- Developed a ROS node for audio feedback and notifications.
- Assisted in the development of the robot's autonomous navigation system.

1.4 Project Scope

The scope of project Hazmat is to design, develop, and deploy an autonomous mobile manipulator capable of detecting, navigating to, and safely handling hazardous materials in indoor industrial environments. The system integrates a mecanum-wheeled base for omnidirectional mobility, a 3-DOF robotic arm with a four-bar linkage for stable manipulation, and a multi-sensor perception suite comprising a RPLiDAR C1, and an Oak-D depth camera for SLAM and object recognition.

The project includes:

- Development of motion control algorithms for precise omnidirectional navigation.
- Implementation of object detection and tracking for hazardous item identification.
- Real-time SLAM integration using LiDAR and depth vision for autonomous localization and mapping.
- Robotic arm control with feedback for precise grasping and placement.
- ROS-based communication framework across subsystems using micro-ROS and a Raspberry Pi 5.
- Power management and autonomy optimization for operation within a 2+ hour runtime window.

Out of Scope:

- Outdoor navigation and GPS-based localization.
- Handling of liquid or airborne hazardous materials.
- Integration with cloud-based monitoring or remote control systems.

2 Background and Theory

2.1 Literature review

2.1.1 Mobile base

- **Advanced Autonomous Platforms:** Some systems employ model-based motion control to achieve precise navigation. For example, a four-wheel-drive mobile robot with wheel encoders was given a tuned PI controller for path tracking, yielding a tracking error under 1.5% in experiments ???. Another design integrates multiple sensors (light, touch, ultrasonic) and utilizes a trapezoidal acceleration-deceleration profile with PID control for smooth, accurate motion [2]. This enabled reliable line-following (e.g. along an S-curve track) and real-time obstacle avoidance on the mobile base.
- **Teleoperated Bases:** Simpler mobile bases often forego autonomous navigation. One wireless robotic arm platform based on Arduino Mega is driven via a PS2 controller, moving forward/reverse and turning in discrete steps [3]. Its performance (speed over distance) was characterized under different battery voltages, showing that insufficient voltage (below 7V) dramatically slows or even halts the robot [3]. Similarly, a bomb-disposal robot employs an RF remote control and a microcontroller to drive a wheeled base [4]. An onboard camera provides the human operator with feedback as the robot moves in all directions and even free-runs;

the base could carry a gripped object up to 1kg [4]. These teleoperated designs are straightforward but rely entirely on human guidance for navigation.

2.1.2 Manipulator

- **Integrated Arms for Pick-and-Place:** Many mobile manipulators incorporate an onboard robotic arm for interacting with objects. One platform uses a 4-DOF arm mounted on a NI sbRIO-controlled base to capture selected objects identified by the vision system and place them at target locations [2]. The arm's joints are driven by DC servomotors, and a PID control law coordinates the arm motion for accuracy. Experimental tests showed the arm moving in multiple axes (up/down, forward/back) to reach and grasp the object once the base was in position [2]. Another system features a 3-DOF manipulator (plus gripper) on a mobile platform, where the end-effector trajectory is planned using inverse kinematics. After the base performs path tracking to the work site, the manipulator's position kinematics (with a shoulder offset) are solved to execute the pick-and-place task precisely.
- **Servo-Based Arms and Payload Limits:** In a more minimalist design, a wireless mobile robot arm built on Arduino uses a 4-DOF revolute servo arm for basic pick-and-place [3]. While effective for light objects, its payload is limited - tests showed it can lift 50-150 g, but at 200 g the servos could no longer function properly [3]. By contrast, a remotely operated EOD robot implemented a 3-DOF arm capable of gripping objects up to 1 kg weight [4]. This arm, controlled via a PIC microcontroller, was observed through a camera and maneuvered by an operator to safely dispose of dangerous items [4]. The lower degree of freedom and direct remote control simplified the arm's design while still accomplishing the essential task.

2.1.3 SLAM (Simultaneous Localization and Mapping)

- **Structured Navigation vs. SLAM:** Some mobile robots avoid full SLAM by using structured environment cues. For instance, one system notes the difficulty of rebuilding an unknown map in real-world complexity and instead focuses on efficient motion planning [2]. It uses line-following (via IR sensors) and known landmarks to navigate to targets, trading autonomy for simplicity. This approach works well in constrained or known environments but doesn't generalize beyond prepared trajectories.
- **2D LIDAR SLAM Techniques:** For robots that do explore unknown areas, 2D LIDAR-based SLAM algorithms are common. A direct comparison of popular methods found that GMapping (particle-filter grid mapping) sometimes failed to produce a consistent map, whereas Hector SLAM and Google Cartographer yielded accurate maps that matched ground-truth

trajectories within a few centimeters [5]. In fact, Cartographer’s real-time loop closure approach (using scan-to-submap matching) enabled large indoor areas to be mapped (tens of thousands of m^2) with high fidelity on limited computing hardware [6]. These laser SLAM systems publish maps and pose estimates that can guide navigation for a mobile base.

- **Vision-Based SLAM:** Monocular visual SLAM has also been explored, though it poses challenges in practice. Tests with ROS-based monocular SLAM on a mobile robot showed issues like scale ambiguity (no inherent distance scale) and heavy computation requirements - the processing had to be offloaded to an external PC, and manual adjustments were needed to fix scale. This indicates that without depth sensors, additional measures are required for reliable vision-only SLAM. Other feature-based approaches use environmental features to aid SLAM; for example, an EKF-SLAM algorithm representing walls and corridors as line segments was validated in a real indoor environment [7]. By extracting line features from laser scans and updating their parameters in the filter, the robot simultaneously localized itself and built a compact line-based map.
- **No-SLAM Teleoperation:** It’s worth noting that teleoperated systems dispense with SLAM entirely. Robots under direct remote control rely on the operator’s situational awareness (often via on-board video feed) for navigation [4]. This eliminates the need for onboard mapping or localization algorithms, simplifying the design at the cost of requiring constant human input and a reliable communication link.

2.1.4 Single Board Computer

- **Embedded Controller Platforms:** Advanced mobile manipulators often use a dedicated single-board computer or embedded controller to handle all the subsystems. In one design, a National Instruments sbRIO-9606 (branded as a KNR controller) serves as the brain of the robot[2]. This real-time embedded platform integrates sensor I/O, motor drivers, and even vision processing capabilities, allowing the robot to process camera images and control motors in a closed-loop without a separate PC. The KNR controller operates as an independent system, reading encoder feedback and running algorithms (like Otsu thresholding and template matching for object tracking) on-board. Such an SBC provides robust performance and timing for coordination of the mobile base and arm.
- **Microcontroller-Only Designs:** In contrast, some robots are built without a traditional SBC by leveraging microcontroller boards. The wireless mobile robotic arm, for example, is driven purely by an Arduino Mega 2560 microcontroller [3]. The Arduino interfaces directly with the wireless receiver and outputs control signals to motors, effectively acting as both the high-level and low-level controller. This greatly simplifies the architecture and reduces cost/power, but it means the system has limited comput-

ing power — it cannot easily perform heavy tasks like vision processing or SLAM on-board. Likewise, the bomb-disposal robot relies on a PIC microcontroller for all decision-making [4], indicating no onboard OS or computer beyond the microcontroller; the video transmission is handled by a separate analog/digital link to the operator, not by an onboard computer. These designs show that an SBC isn't always necessary when the task scope is limited.

- **Offloading Computation:** When higher processing capability is needed, designers may incorporate a more powerful SBC or offload computation to an external system. In one SLAM evaluation, for instance, the robot's sensor data was sent to a laptop for processing because the on-board controller could not handle the intensive monocular SLAM algorithms. This highlights a trade-off: using a capable SBC (like a Raspberry Pi, Jetson, or an FPGA-based board) can enable advanced features on the robot (mapping, complex vision), but it increases power consumption and integration complexity. Each project balances these factors differently depending on requirements.

2.1.5 Microcontroller

- **Real-Time Low-Level Control:** Microcontrollers play a crucial role in the real-time control of mobile manipulators. They handle tasks such as sensor reading, motor actuation, and ensuring timely responses. In the Arduino-based mobile arm, the ATmega2560 microcontroller generates PWM signals to drive the arm's servos and reads inputs from the wireless game controller. The simplicity of toggling I/O pins at microsecond resolution makes microcontrollers ideal for precise control of motors (e.g., maintaining servo positions or motor speeds). Even in systems with an SBC, microcontroller units (or microcontroller-like cores) may be present for hard real-time tasks. For example, the NI sbRIO's onboard processor and FPGA take on microcontroller duties by interfacing with encoders and sensors directly, closing feedback loops at high speed while the higher-level logic runs in parallel.
- **Distributed Control Architecture:** Some designs use a single microcontroller to manage both the base and manipulator, while others might distribute tasks. In the dangerous-object disposal robot, a single PIC microcontroller (programmed via MPLAB) handles all motion - it parses the decoded remote commands and issues signals to both the drive motors and the arm's servos [4]. This central unit orchestrates wheel movements and gripper actions in sync, proving sufficient for a teleoperated scenario. In more complex robots, one might find multiple microcontrollers (for example, one dedicated to the arm and one to the base) communicating with an SBC. However, increasing the number of controllers requires careful coordination. Overall, microcontrollers are favored for their reliability and deterministic control, especially in safety-critical manipulations,

whereas computation-heavy tasks are offloaded to higher-level processors when needed [5].

2.1.6 Pros and Cons of Different Methods

- **Autonomy vs. Teleoperation:** Fully autonomous mobile manipulators can navigate and perform tasks without human intervention, which is crucial in inaccessible or dynamic environments. They leverage SLAM and path planning to handle unknown areas (e.g., using LIDAR SLAM to map and localize with 3 cm accuracy [5]). The trade-off is significant system complexity: mapping a real-world environment is challenging [2] and requires robust algorithms and sensor fusion. Teleoperated robots, on the other hand, avoid this complexity by relying on human control. Designs like the PS2-controlled Arduino robot or the RF-controlled bomb disposal unit are far simpler - no SLAM or AI is needed [4]. The downside is they require continuous operator attention and a reliable communications link, and their effectiveness is limited by human reaction time and line-of-sight (or camera view) of the scene. In summary, autonomy increases capability and independence, but teleoperation offers simplicity and ensures a human is in the loop for critical decisions.
- **Control Strategies and Complexity:** The choice of control strategy affects both performance and development effort. Advanced control methods (e.g. model-based controllers, inverse kinematics, or even model predictive control) can greatly enhance precision and efficiency. For instance, incorporating kinematic modeling and feedback allowed one mobile base to track paths with only 1.41% error ??, and using inverse kinematics enabled accurate arm positioning ?. However, these methods demand careful system identification, tuning, and computational resources. Simpler control approaches like PID loops or open-loop teleoperation are much easier to implement and require less processing power. They can achieve adequate performance (the NI-based robot effectively followed lines and positioned its arm with basic PID control), but they might yield suboptimal motion (e.g., more overshoot or slower response) compared to optimally tuned model-based controllers. Thus, engineers must balance the need for precision against the available time and hardware for implementing complex controls.
- **Hardware and Integration:** Using a high-end integrated controller versus a basic microcontroller board presents a clear trade-off. Integrated SBC solutions (such as the sbRIO or a mini-PC) provide a one-stop platform to handle vision, navigation, and control with ease of integration [2]. They often come with rich development tools (LabVIEW, ROS, etc.) and can run sophisticated algorithms (object recognition, global planning) on-board. The cons are higher cost, greater power consumption, and typically increased weight/size. Conversely, microcontroller-centric builds (Arduino, PIC) are low-cost, lightweight, and energy-efficient [3],

but they lack the horsepower for computation-heavy tasks. In practice, some projects augment microcontrollers with external computing (e.g., streaming sensor data to a laptop for processing) or use hybrid setups (microcontroller for real-time control, SBC for higher-level processing). The optimal architecture depends on the application: a research prototype might afford a powerful onboard computer, whereas a field-deployed simple robot might favor minimal electronics for robustness.

- **Actuation and Payload Considerations:** Different approaches to actuation lead to different capabilities. Hobby-grade servos (as seen in the Arduino-based arm) are inexpensive and easy to control but impose strict payload limits (0.15 kg in that design before stalling [3]). They are suitable for educational or light-duty robots but not for heavy manipulation. Upgrading to larger motors or professional servos increases payload (the EOD robot arm handled 1 kg [4]) and speed but requires more powerful drivers and power supply and often closed-loop control for precision. Moreover, a heavier arm can destabilize a small mobile base when extended; this is why careful mechanical and control design (like lowering speed or using a counterbalance) is needed for heavier payloads. Choosing the right actuators involves balancing the desired payload and reach of the arm against the battery capacity, weight budget, and cost constraints of the overall system.
- **Sensor Suite and Environmental Perception:** The methods each system uses to perceive its environment come with pros and cons. A robot equipped with LIDAR and cameras can perform complex tasks like SLAM and object recognition, enabling true autonomy in diverse environments. The NI-based platform, for example, used a camera for vision (with color tracking and template matching) and multiple ultrasonic rangefinders for obstacle detection, giving it a rich awareness of its surroundings. The downside is the integration effort and processing needed for these sensors. Simpler robots that use only basic sensors (bump switches, IR line sensors, a few sonars) are much easier to build and program, but they can only operate in structured settings (following lines, avoiding obstacles directly ahead). If a task demands high reliability in unstructured or dynamic environments, investing in a robust sensor suite is worthwhile; otherwise, a lean sensor setup keeps the system affordable and less complex.
- **Experimental Validation and Results:** A key advantage of hardware-focused research is the ability to validate methods in real-world tests. Different approaches have been proven through experiments: the four-wheel mobile base demonstrated its path tracking accuracy on a physical track ??, and the wireless arm's trials quantified the relationship between battery voltage and speed [3] as well as its payload capacity. SLAM algorithms were not just simulated but also tested on a robot doing laps in an indoor environment, revealing practical differences between mapping methods [5]. Such empirical data is invaluable - it highlights failure modes (e.g., GMap-

ping’s map inconsistency [5], or the Arduino arm stalling at 200 g [3]) and verifies that certain methods meet their performance goals (e.g., Cartographer’s real-time mapping success [6]). In summary, each method comes with trade-offs, and examining the engineering decisions and outcomes across projects provides guidance for future designs. The best solution for a mobile manipulator will depend on the specific use-case requirements, whether it’s the autonomy level, precision, payload, or operating environment that is the top priority.

2.2 Motor Control

2.2.1 Mecanum wheels

Mecanum wheels allow a robot to move in any direction without changing its orientation. Each wheel has angled rollers (at 45°) mounted around its rim. When the wheels spin, they create a combination of forward/backward movement and sideways movement. By carefully controlling the speed and direction of each wheel, the robot can move forward, backward, sideways (strafe), diagonally, or rotate on the spot. This gives the robot full omnidirectional mobility, as outlined in Figure 2.

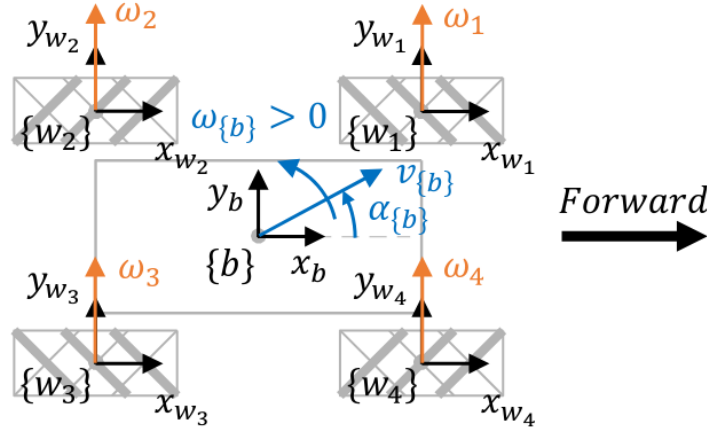


Figure 2: A common 4-wheeled mecanum configuration

2.2.2 Wheel encoders

The mecanum wheels come along with an encoder for each wheel and the encoders operate through hall effect sensors. A hall effect sensor works by detecting the magnetic field from a magnet ring attached to the wheel shaft. As the wheel turns, the magnet ring spins with it, causing the magnetic field near the hall sensor to change. Every time the sensor detects a change in the

magnetic field, it creates an electrical pulse. These pulses are then sent to the microcontroller to find out the wheel RPM.

To calculate the RPM, the motor's encoder pulses were counted over a fixed period. Each full revolution generates a known number of pulses (20 pulses per revolution in our motor). The pulse count was divided by the number of pulses per revolution to determine the number of revolutions per second and then multiplied by 60 to convert it into revolutions per minute (RPM). After each calculation, the pulse count was reset for accurate measurement in the next cycle.

2.2.3 PID control

A PID controller (Proportional-Integral-Derivative controller) is used in a closed-loop feedback system to control the outer loop of the robot. In this system, the robot's current joint position is continuously measured. The error is calculated by comparing the current position to the desired position. The PID controller then processes this error to generate a control input that tries to minimize it: the proportional (P) part reacts to the size of the error, the integral (I) part reacts to how long the error has been there (accumulated error over time), and the derivative (D) part reacts to how fast the error is changing. The PID controller continuously adjusts the robot's movement based on this feedback, aiming to keep it closely following the target trajectory. The following equations describe this controller mathematically, and Figure 3 outlines it as a block diagram,

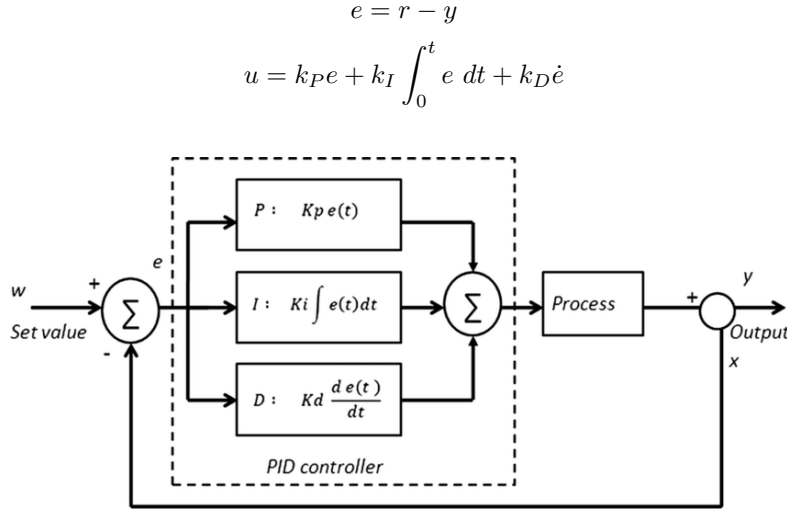


Figure 3: Block diagram of PID control

2.2.4 PWM control

A PWM (Pulse-Width Modulation) is a method used to “convert” digital signal to analog signal by switching on and off the digital output. 3 main concepts of PWM:

- (P) - Pulse, the digital signal 0 (LOW) and 1 (HIGH),
- (W) - Width, the duration of HIGH signal,
- (M) - Modulation, which is the duty cycle which adjusts the duration to control output power.

2.3 Sensing

2.3.1 Odometry

Odometry is the process of estimating a robot’s change in position (x, y) and orientation (θ) by integrating its measured velocities over time. In wheeled robots, we typically derive those velocities from the wheel encoders’ readings i.e. RPM values in our case.

The process of computing the robot’s motion begins with the acquisition of RPM values from the wheels. These RPM values are first converted into angular velocities to represent the rotational speed of each wheel in radians per second. Once the angular velocities ($\omega_{fl}, \omega_{fr}, \omega_{rl}, \omega_{rr}$) are obtained, the forward kinematics equations are applied to determine the robot’s linear velocities in the x and y directions, as well as its angular velocity about the z-axis, given the radius of the wheels r , and length l_x and width l_y from the 2D centre of the robot

$$\begin{aligned}v_x &= \frac{r}{4}(\omega_{fl} + \omega_{fr} + \omega_{rl} + \omega_{rr}) \\v_y &= \frac{r}{4}(-\omega_{fl} + \omega_{fr} + \omega_{rl} - \omega_{rr}) \\ \omega_z &= \frac{r}{4(l_x + l_y)}(-\omega_{fl} + \omega_{fr} - \omega_{rl} + \omega_{rr})\end{aligned}$$

Following the computation of the velocities, integration over time is performed to update the robot’s position and orientation. This integration yields the robot’s displacement along the x and y, and orientation along z, effectively estimating its trajectory over time based on the measured motion data

$$\begin{aligned}x &= \int v_x dt \\y &= \int v_y dt \\ \theta &= \int \omega_z dt\end{aligned}$$

2.3.2 Localisation and Mapping

Localisation is a fundamental ability required by robots for autonomous navigation, as it involves estimation of the robots pose (position and orientation) in the same frame of reference as its goal. The other component is mapping, which attempts to create a map of the obstacles and areas of interest to the robot, which is crucial to allow for path planning with obstacle avoidance. This gives way to the common problem tackled in robotics known as simultaneous localisation and mapping (SLAM), which attempts to approximate both in a computationally feasible ways. Since all quantities in this problem are stochastic and discrete, the objective is to compute the distribution of the robot state, x_t , the map of the environment, m_t , given a series of observations, $o_{1:t}$, and control inputs, $u_{1:t}$,

$$P(m_{t+1}, x_{t+1} | o_{1:t+1}, u_t)$$

Using Bayesian computation, the posterior distributions of each x_t and m_t can be calculated separately when given the value of the other. Many statistical algorithms exist to tackle this problem, like the Extended Kalman Filter and sequential Monte Carlo method, as well as graph-based methods. Amongst these the sequential Monte Carlo method, also known as the particle filter, is most popular in mobile robots. It models the states and observations as state-space equations given respectively as

$$\begin{aligned}x_t &= g(x_{t-1}) + W_{t-1} \\ o_t &= h(x_t) + V_t\end{aligned}$$

where W_t and V_t are mutually independent, and the equations are sequentially solved using a number of different Bayesian methods depending on the constraints of the specific system.

3 Methodology and Implementation

In the current era of rapid technological advancement, the shift toward autonomous systems has become not only logical but essential. Automation reduces human risk exposure, optimizes operational efficiency, and expands capabilities beyond biological limitations. Tasks involving hazardous materials present significant health and safety concerns for human operators. Exposure to toxic compounds, unstable environments, and unpredictable variables necessitates the development of non-human agents capable of executing such operations with precision and resilience.

This chapter outlines the design methodology and implementation sequence of an autonomous hazardous waste retrieval unit. The system has been engineered for deployment in environments deemed unsafe for prolonged human

presence. Operational goals include waste identification, secure collection, and optimized navigation, all executed without direct human intervention.

3.1 Concept Development and Evaluation

Our primary novel concept was the manipulator, hence we considered multiple designs, and used a design choice matrix to find the most appropriate one. Our criterion were: the appropriateness and the end-effector to our task and its simplicity; the accuracy that could be achieved in end-effector positions; the static and dynamic (during movement of joints or mobile base) stability of the manipulator; the load it could carry; and finally the complexity of its implementation.

The designs we considered as follow, and their decision matrix scores are shown in the table below,

- A 2-DOF forklift that would require objects to be placed at some elevation and would function with two linear actuators that would move the hoist horizontally in the direction of the robots movement and vertically up or down.
- A suction gripper that would be limited to working on flat surfaces, and would have three revolute joints for a full range of motion in 3D space.
- A crane mechanism with one rotating joint around the z (vertical) axis and one linear actuator to lower or raise the hoist to be able to reach and hold the objects in a safe place.
- A simple 3-DOF arm with only revolute joints and a two-claw grippers was chosen as a baseline for the rest. It concluded as the best choice and was further developed with a four-bar linkage mechanism to further improve stability by bring the manipulator's centre of the mass lower.

Gripper type	Weight	Forklift	Suction gripper	Crane	Regular 3-DOF arm
Degrees of freedom	0	2	3	3	3
End effector	25	1	2	2	3
Accuracy/tolerance	25	5	4	3	3
Stability & weight	25	1	3	2	3
Strength & load capacity	15	5	1	3	3
Electrical & programming complexity	10	4	3	3	3
Final score	0	290	270	250	300

3.2 Digital Design

The diagram in Figure 4 illustrates an FSM diagram of our robot's logic and operation.

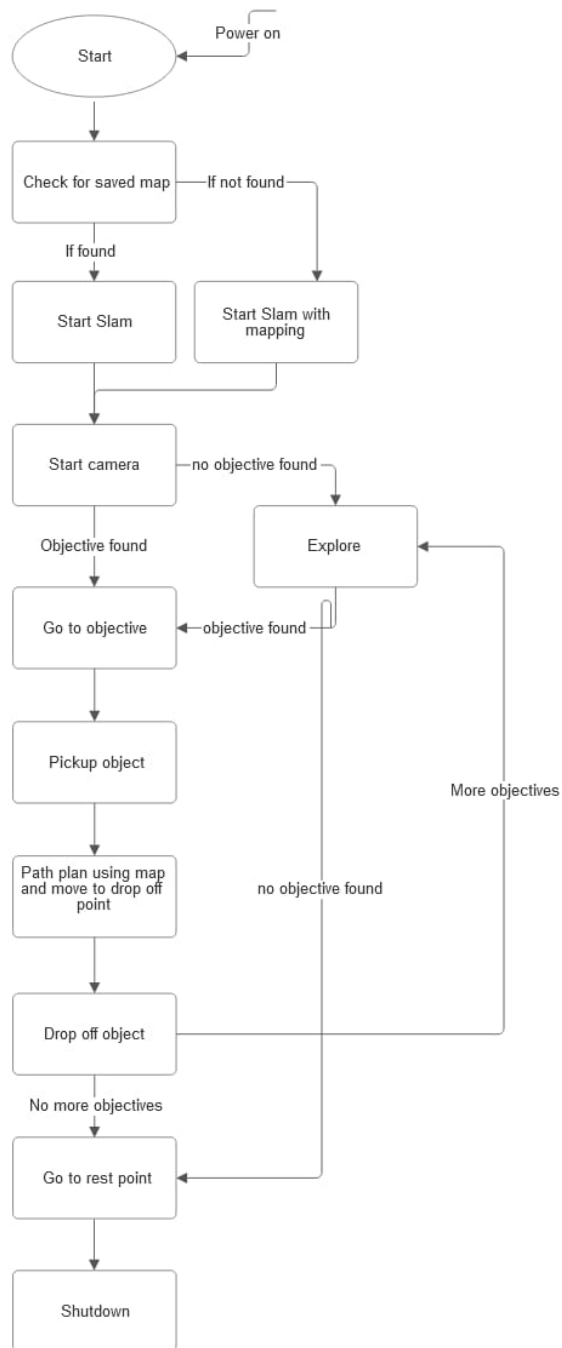


Figure 4: FSM diagram of our robot's logic

3.3 Mechanical Design

3.3.1 Mobile Base

The base plate is a flat, 3D-printed PLA structure designed to support the primary electronic and mechanical components of the robot. It features mounting holes for the motors and mecanum wheels, along with embedded M3 heat-set inserts to ensure secure fastening of the motor brackets, depicted in Figure 5. Adjacent to each motor mount is a 20mm * 6mm rectangular cutout, specifically designed for routing motor cables from the underside of the plate. A larger central slot is incorporated to accommodate additional wiring, including battery leads and power distribution cables. The Osoyoo motor driver mount, located at the center of the base plate, also includes M3 heat-set inserts for reliable installation. Mounting provisions are included for the Raspberry Pi 5, utilizing M2.5 heat inserts, as well as for the ESP32-S3, RPLIDAR, and buck converter. The accompanying engineering drawing in Figure 6.

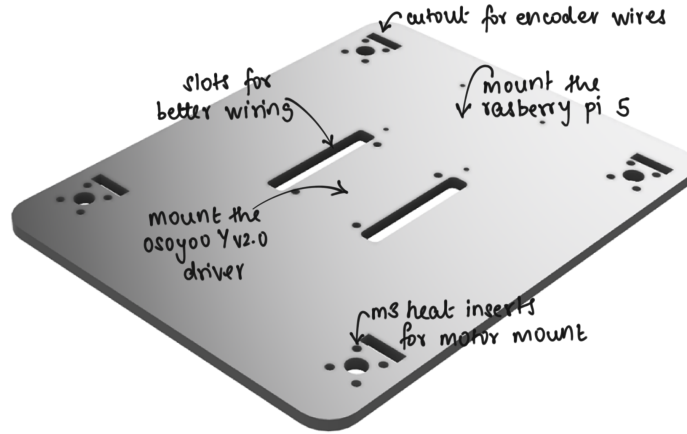


Figure 5: 3D render of the base plate

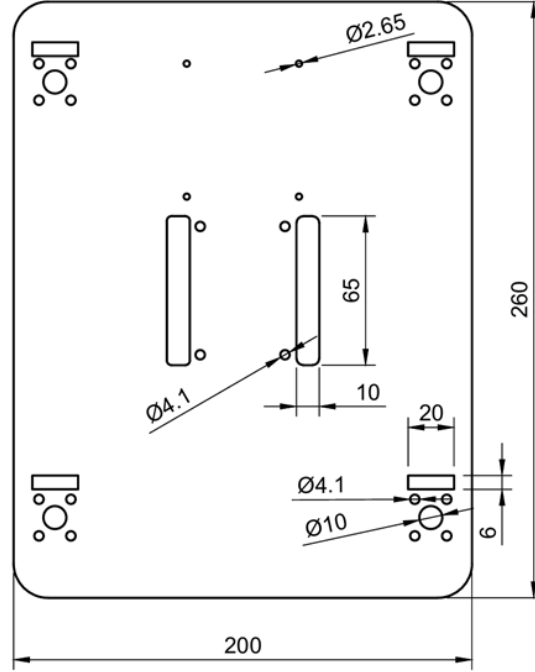


Figure 6: Engineering drawing of the base plate

The top plate is a flat, 3D-printed PLA panel engineered to support the manipulator arm and sensor payload. It incorporates an integrated mount for the arm base, along with a dedicated slot to accommodate a slip ring, enabling continuous 360-degree rotation of the arm without cable entanglement. The top plate is elevated above the base plate via standoffs, which provide sufficient vertical clearance to house the onboard camera and ensure an unobstructed field of view for the RPLIDAR. A designated camera mounting point is also included in the design. The standoff configuration has been optimized to maximize the LiDAR's coverage while maintaining structural integrity. The base servo motor is mounted through the top plate, forming the foundation for the 3-DOF manipulator arm, which is subsequently assembled on top to enable rotational movement around the base axis. Figures 7 and 8 illustrate the top plate, and Figure 9 contains the engineering drawing.

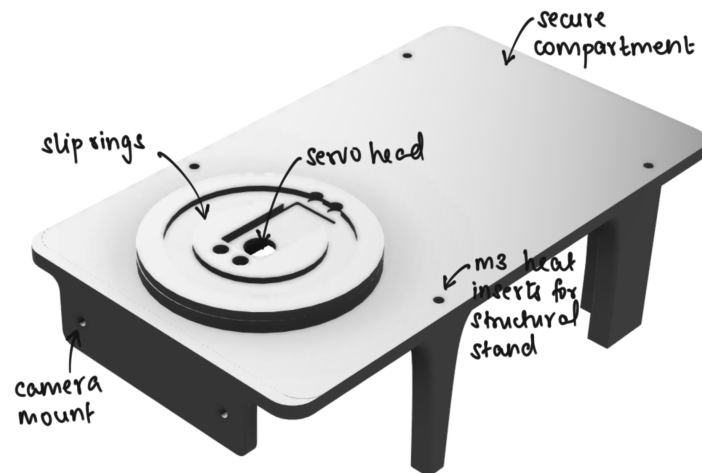


Figure 7: 3D render of the top plate

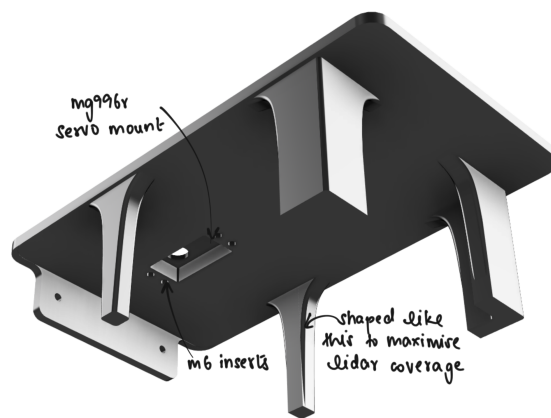


Figure 8: Another view of the top plate

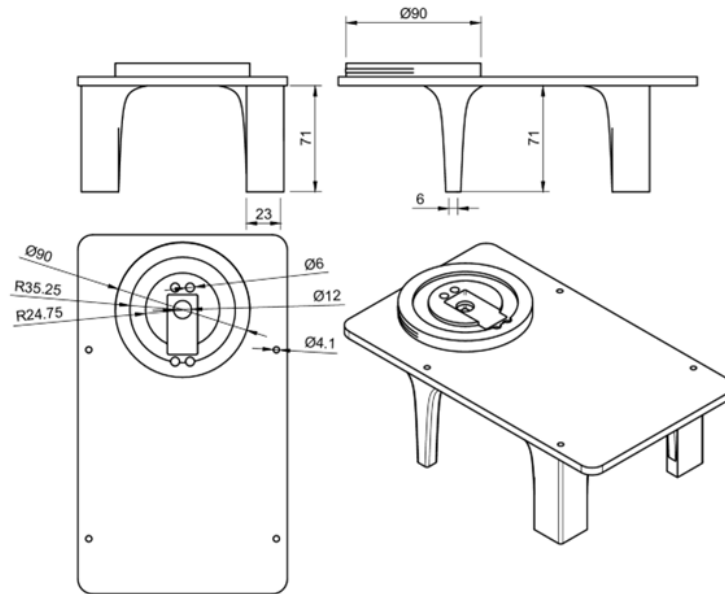


Figure 9: Engineering drawing of the top plate

A secure payload compartment is mounted on the top plate to enable safe and stable transportation of hazardous waste between collection and disposal points, as shown in Figures 10 and 11.

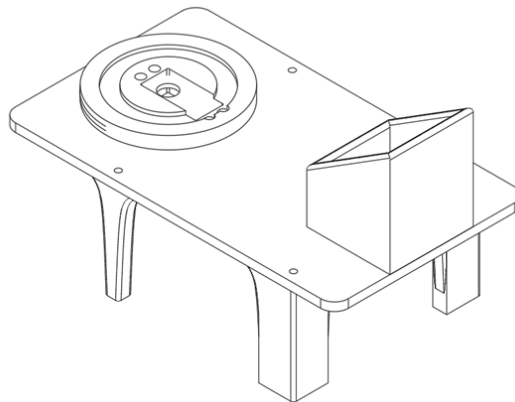


Figure 10: Top plate with payload compartment

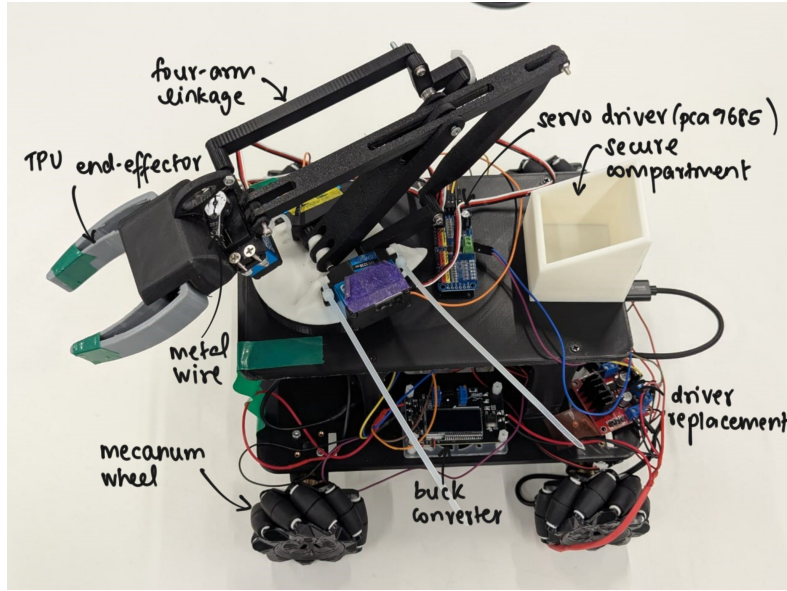


Figure 11: The final assembly

3.3.2 Manipulator

The manipulator arm features three degrees of freedom (3-DOF) and incorporates an offset four-bar linkage mechanism. The first joint enables rotational motion about the z-axis, while the second and third joints provide rotational movement about the x-axis. Notably, the third joint is actuated via a four-bar linkage connected to a servo motor mounted at the base of the arm. This configuration minimizes the mass and rotational inertia acting on the distal segments of the arm, thereby enhancing load-bearing capability and overall system stability. By offloading the actuation to the base, the design significantly improves mechanical efficiency and reduces strain on the upper servos. Figures 12 and 13 respectively contain an illustration and the engineering drawings for the arm.

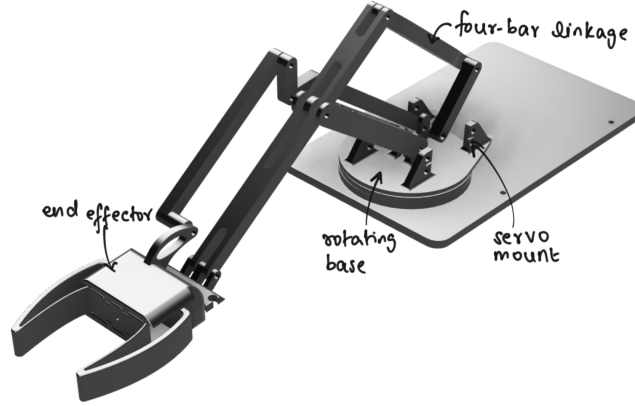


Figure 12: 3D render of the base arm

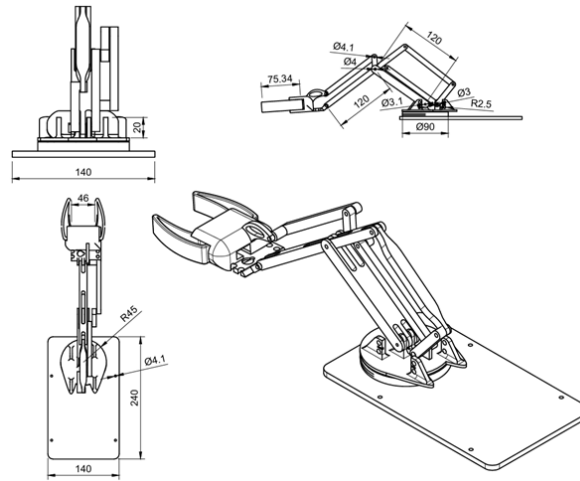


Figure 13: Engineering drawing of the arm

The manipulator is maintained in an upright orientation by a double four-bar linkage system. The manipulator assembly consists of five components: the housing, the flexible gripper, the gripper end holder, the wire, the servo pulley, and the servo motor.

The housing, gripper end holder, and servo pulley are 3D-printed using PLA, while the flexible gripper is fabricated using TPU filament for its elasticity. This design was selected for its simplicity, reliability, and ease of manufacturing. The

table below details the Denavit-Hartenberg parameters for the arm.

i	a_i	α_i	d_i	θ_i
1	0	$\pi/2$	0	θ_1
2	120mm	0	0	θ_2
3	120mm	0	0	θ_3

3.3.3 Assembly

The robot assembly comprises five primary components: the base plate, standoffs, top plate, manipulator arm, and associated electronics.

Step 1: Base Plate Assembly

1. Print the base plate using suitable material (e.g., PLA, PETG, or ABS).
2. Mount the four motors to the underside of the base plate using four M3 screws per motor (16 M3 screws total).
3. Attach the mecanum wheels to the motor shafts and secure each wheel with the appropriate fastener.

Step 2: Electronics on the Base Plate

1. Mount the Raspberry Pi 5 onto the designated holes on the base plate using four M2.5 screws.
2. Attach the buck converter to its allocated position using double-sided tape.
3. Secure the Osoyoo Model Y v2.0 motor driver at the center of the base plate using M3 screws.
4. Mount the Rplidar opposite the Raspberry Pi.
5. Install the ESP32-S3 module opposite the buck converter.

Note: All mounting holes include heat-set inserts for improved screw retention and secure fastening.

Step 3: Wiring and Power Distribution

1. Route the motor wires to the ESP32-S3 through the two slots adjacent to the Osoyoo driver.
2. Secure the battery beneath the base plate using two zip ties through the same slots.
3. Connect power lines from the battery to:
 - The Osoyoo Model Y v2.0 motor driver.
 - The buck converter.
4. Distribute regulated power from the buck converter to the rest of the electronics, including the Raspberry Pi and ESP32-S3.

Step 4: Structural Assembly

1. Install four standoffs on the base plate using M3 screws, these standoffs provide vertical clearance for the components and for the lidar to function correctly.
2. Attach the top plate to the standoffs and secure temporarily.
3. Apply super glue to the base of each standoff to bond them to the base plate for structural rigidity.
4. Once the glue has fully cured, the top plate can be unscrewed to allow access to the internal components without damaging the assembly.

Step 5: Top Plate Assembly

1. Mount the base servo underneath the top plate at the specified position.
2. Assemble the 3-DOF four-bar linkage manipulator arm on the top plate, interfacing it with the base servo.
3. Attach the secure payload compartment.
4. Attach the Oak-D camera on the top plate along with the servo driver.
5. Complete all wiring from the manipulator servos to the Raspberry Pi or ESP32-S3 as per the control logic.

3.4 Sensors

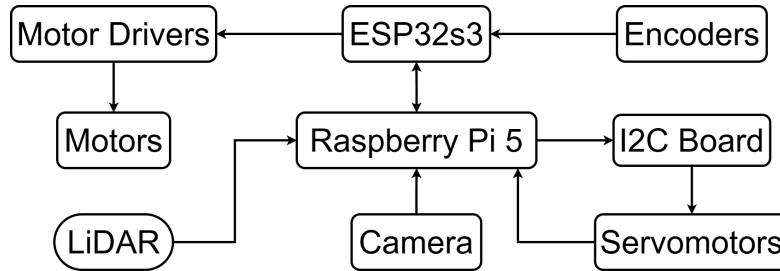


Figure 14: Data flow diagram

Hazmat's arm has a 3-DOF arm with 3 servomotors (hereafter referred to as servos). We decided to create an inverse kinematics system, and for that, we need a feedback angle from the servos. Our servomotors had a potentiometer inside that outputs an analog signal that is correlated to servomotor's actual angle, visible in Figure 15. In order to get that signal servos were opened, and additional wire was soldered to one of the potentiometers pins, as shown in Figure 16.

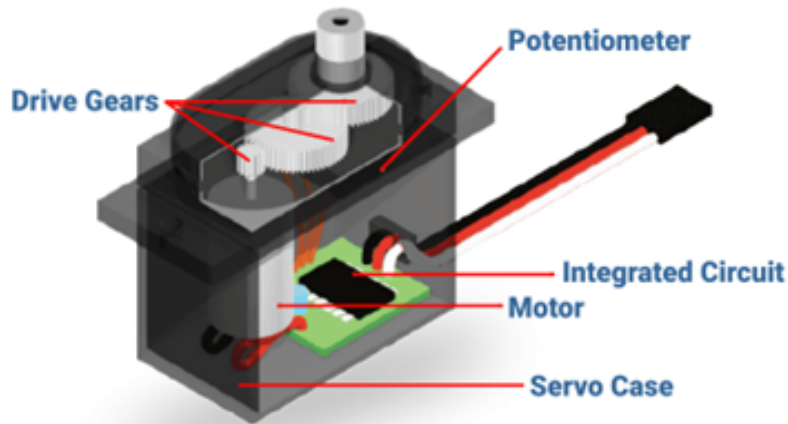


Figure 15: Servomotor working principle



Figure 16: Servomotor with orange wire soldered to potentiometer

Hence, the fourth wire was outputting feedback signal. This signal had minimum and maximum values that were corresponded to the minimum and maximum servo angles. The range of the analog signal differs in between different servo; it was depended on the servo calibration method when they were produced. In the code, it was mapped to correspond with an angle. The rest of our sensors connections are illustrated in Figure 14.

3.5 Software

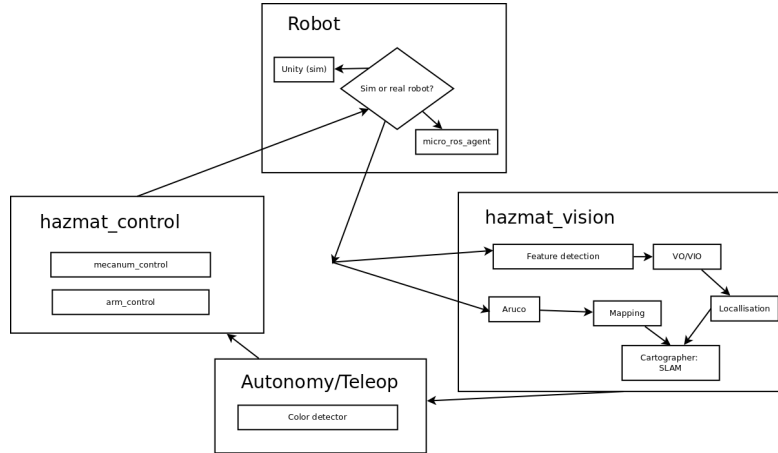


Figure 17: High level overview of the ROS nodes and topics

Our software stack was primarily built upon the expansive ecosystem of Robot Operating System (ROS) [1], which provided cross-script communication, parallelisation, and ready-to-use packages maintained by robotics companies and industry experts, out of the box. Figure 17 shows an overview of our ROS nodes and data-flow.

3.5.1 SLAM

The robots transform tree was carefully structured in the Universal Robot Description File (URDF) to support Cartographer’s configurations. The IMU ([imu_link](#)) and LiDAR ([scan_link](#)) were each fixed relative to the central robot frame ([base_link](#)). The overall transformation (TF) hierarchy followed the structure shown in Figure 18.

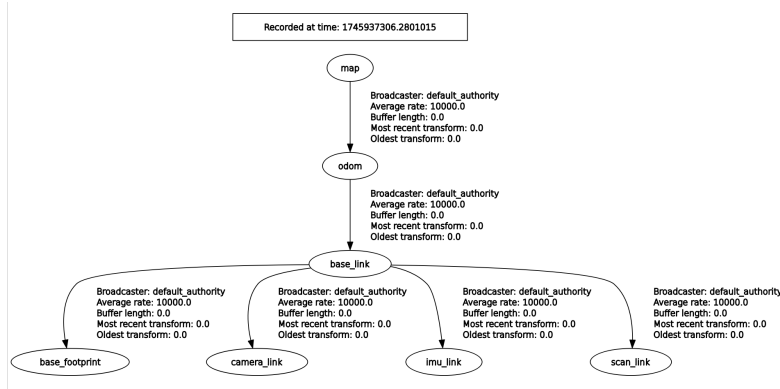


Figure 18: Transformation tree visualised with rqt_tf_tree

Our SLAM solution was built upon the ROS packages for Google’s Cartographer library, configured for 2D mapping using LiDAR and IMU data. The core configuration for Cartographer was defined in the Lua script (cartographer.lua), where key parameters such as `use_trajectory_builder_2d = true` and `use_imu_data = true` were set to enable 2D SLAM operation and integrate IMU data into the state estimation pipeline. The IMU played a crucial role in providing continuous orientation and rotational velocity data, improving pose estimation accuracy, particularly during rapid movements or in environments with sparse LiDAR features. The following listing is our lua configuration file for Cartographer:

```

include "map_builder.lua"
include "trajectory_builder.lua"

options = {
  map_builder = MAP_BUILDER,
  trajectory_builder = TRAJECTORY_BUILDER,
  map_frame = "map",
  tracking_frame = "imu_link",
  published_frame = "imu_link",
  odom_frame = "odom",
  provide_odom_frame = true,
  publish_frame_projected_to_2d = true,
  use_odometry = false,
  use_nav_sat = false,
  use_landmarks = false,
  publish_to_tf = true,
  num_laser_scans = 1,
  num_multi_echo_laser_scans = 0,
  num_subdivisions_per_laser_scan = 1,
  num_point_clouds = 0,

```

```

lookup_transform_timeout_sec = 0.2,
submap_publish_period_sec = 0.3,
pose_publish_period_sec = 5e-3,
trajectory_publish_period_sec = 30e-3,
rangefinder_sampling_ratio = 1.,
odometry_sampling_ratio = 1.,
fixed_frame_pose_sampling_ratio = 1.,
imu_sampling_ratio = 1.,
landmarks_sampling_ratio = 1.,
}

MAP_BUILDER.use_trajectory_builder_2d = true

TRAJECTORY_BUILDER_2D.min_range = 0.05
TRAJECTORY_BUILDER_2D.max_range = 3.
TRAJECTORY_BUILDER_2D.missing_data_ray_length = 3.
TRAJECTORY_BUILDER_2D.use_imu_data = true
TRAJECTORY_BUILDER_2D.use_online_correlative_scan_matching = true
TRAJECTORY_BUILDER_2D.motion_filter.max_angle_radians = math.rad(0.1)

POSE_GRAPH.constraint_builder.min_score = 0.65
POSE_GRAPH.constraint_builder.global_localization_min_score = 0.7

```

The LiDAR sensor was responsible for generating 2D laser scans of the surrounding environment. These scans were pre-processed through a custom filtering step implemented in the `scan_callback()` function of the Color node. In this filtering process, a forward-facing sector was isolated (defined between -80° and $+100^\circ$) and invalid or noisy range readings were discarded based on `range_min` and `range_max` conditions. The filtered data was published on the `/scan_filtered` topic, which was then used by the Cartographer SLAM node for map building. Cartographer constructed an occupancy grid map by matching these incoming filtered scans against the existing map. The following listing shows the code for `scan_callback`:

```

def scan_callback(self, msg: LaserScan):
    scan = msg
    self.scan_data = msg
    # Define the front sector in radians
    front_sector_min = -math.radians(80) # -0.5236 rad
    front_sector_max = math.radians(100) # 0.5236 rad

    # Calculate the index range corresponding to the front sector
    # LaserScan angles: angle = scan.angle_min + index * scan.angle_increment
    start_index = int((front_sector_min - scan.angle_min) / scan.angle_increment)
    end_index = int((front_sector_max - scan.angle_min) / scan.angle_increment)

```

```

# Ensure indices are within valid bounds
start_index = max(0, start_index)
end_index = min(len(scan.ranges) - 1, end_index)

# Extract the ranges within the desired sector and filter valid readings
front_ranges = [r for r in scan.ranges[start_index:end_index+1]
if scan.range_min <= r <= scan.range_max]:
    if front_ranges:
        self.closest_distance = min(front_ranges)
    else:
        print("No valid range measurements in the front sector.")

```

To further enhance localization robustness, `use_online_correlative_scan_matching` was enabled in the Lua configuration, and motion filter parameters such as `motion_filter.max_angle_radians` were tuned to suppress noise from minor pose fluctuations. Through the fusion of filtered LiDAR data and IMU measurements, Cartographer achieved more reliable and drift-resistant SLAM performance. While LiDAR provided detailed spatial mapping through scan matching, the IMU contributed to maintaining accurate heading and motion estimation during situations where scan matching alone would have been less reliable.

3.5.2 Vision

The camera was utilised for two purposes: segmenting colours and detecting Aruco markers. The first was done using hue filtering, where the first component of the Hue-Saturation-Value (HSV) representation of the image was used to filter out colours, with the other two were configured to allow for a range of lighting conditions. Regions from the hue-filtered image were then further separated using common contours to detect specific objects in the scene. This was then implemented for two applications:

- following coloured objects in the environment. The minimum size threshold on the contour regions was set to remove noise and ensure only valid targets were processed. The robot was set to move left or right based on where the centre-point of the target was and move towards it until reaching a certain distance away from it, as demonstrated in Figure 19.
- visual servoing for the manipulator's base joint to rotate and align itself with the object to be picked up. The manipulator's end-effector was coloured distinctively to achieve tracking of both ends, and the operation is demonstrated in Figure 20.

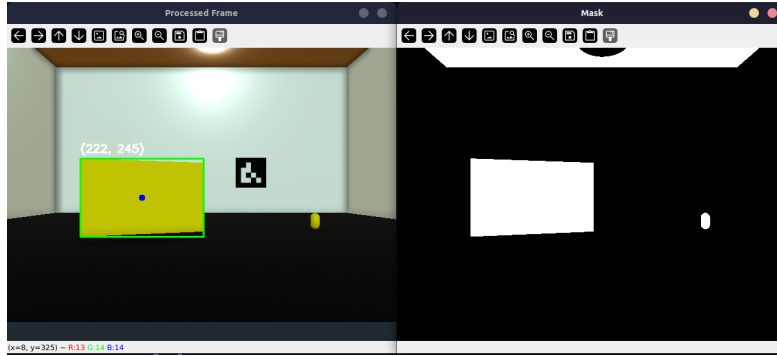


Figure 19: Yellow target detection

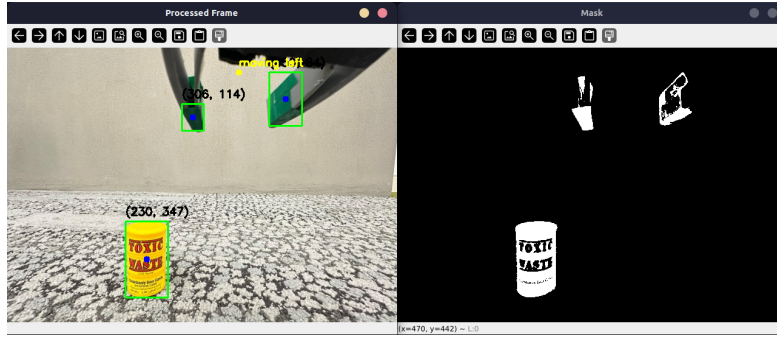


Figure 20: HSV mask and detection of object and gripper

3.5.3 Embedded systems

The ESP-32 were programmed with the ESP-IDF framework and the micro-ROS libraries to facilitate communication between it and the Pi. Our custom message `MecanumCmd` was compiled and we setup subscribers using it for commanding our wheels, as well as a publisher for the wheel encoder values. The transport we used was serial with the maximum baud-rate of 115200, as it provided less latency than anything wireless and was possible due to an abundance of USB ports on the Pi.

3.5.4 Control

To command the mobile base, we computed the inverse kinematics (IK) of the robot to give us individual wheel velocities. These were then supplied as target velocities to a PID controller per-wheel, to ensure they were followed in stable manner. The IK equations are given as follows

$$\begin{aligned}\omega_{fl} &= \frac{1}{r}(v_x - v_y - (l_x + l_y)\omega_z) \\ \omega_{fr} &= \frac{1}{r}(v_x + v_y + (l_x + l_y)\omega_z) \\ \omega_{rl} &= \frac{1}{r}(v_x + v_y - (l_x + l_y)\omega_z) \\ \omega_{rr} &= \frac{1}{r}(v_x - v_y + (l_x + l_y)\omega_z)\end{aligned}$$

3.5.5 Audio feedback

An audio playback service node was implemented to enable verbal interaction between the robot and its environment. The node, named `AudioService`, was developed to enable dynamic sound output on the robot through ROS 2 services. It provides two service endpoints: one to start audio playback and another to stop it.

This functionality allows the robot to respond to events or user interactions with verbal cues, alerts, or predefined audio messages and it is triggered when the robot detects a human or obstacle directly in front of it. Upon detection, the robot initiates audio playback for a specified duration before executing a maneuver to navigate around the object. This behaviour allows the robot to provide an audible cue such as a warning or instruction prior to acting, enhancing its interactive and socially aware operation. While this integration of the audio node into the robot’s feedback loop demonstrates effective modular design, it is acknowledged that the timing, responsiveness, and contextual awareness of this feature require refinement in the final prototype.

3.6 Circuit schematics

For the battery, we have used 4-cell 14.8V LiPo battery that gives power to motor driver “OSOYOO Model Y v2.0” and 4 motors. Then, from battery current goes to buck converter that steps it down from 14.8V to 5V.

The converter powers the Pi and I2C (PCA9685) board for the servo motors. Then Pi then powers Lidar Speaker. The simplified diagram presents an easier overview of the current flow in the system. The Raspberry Pi 5 also provides power for additional peripherals like the camera and LiDAR. The ESP32-S3 board handles the encoders to monitor motor speed and movement. The flowchart in Figure 21 helps visualize the high-level power distribution and the logical connections between the major components without diving into the detailed wiring. Figures 22 and 23 show the initial and final schematics of the wiring respectively, where each part is carefully labelled in the schematic to show how power and data are distributed between the motors, encoders, servo motors, sensors, and controllers.

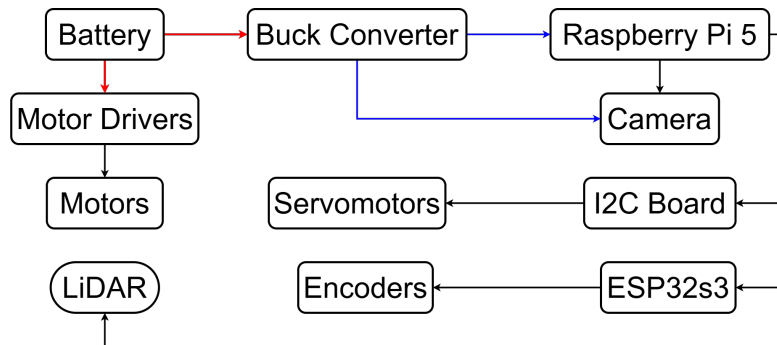


Figure 21: High-level electric circuit diagram

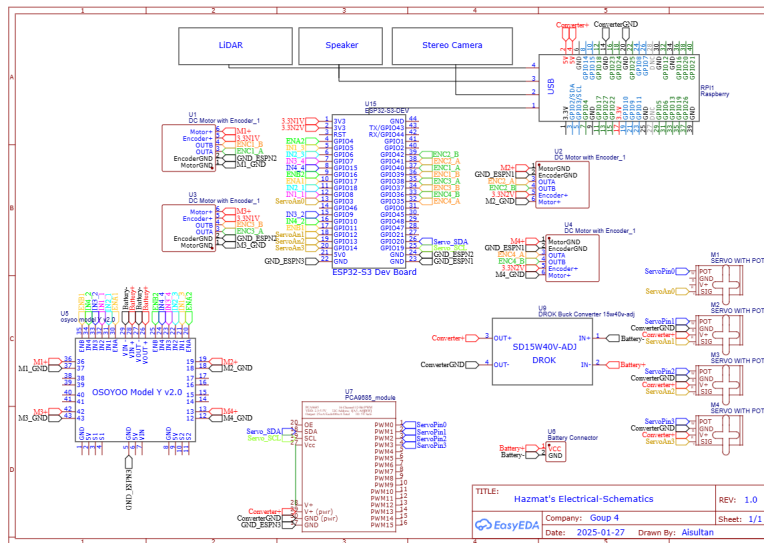


Figure 22: Previous schematics

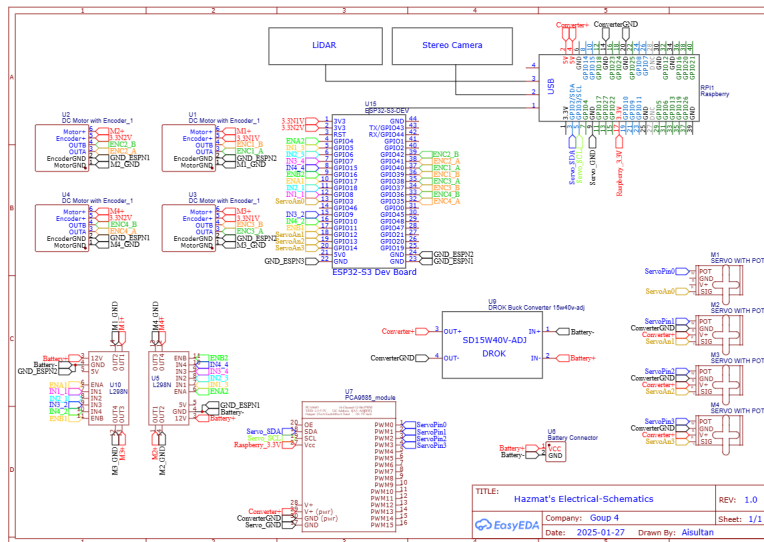


Figure 23: Current schematics

3.7 Prototype development

The development of the robot’s physical structure began with the selection and procurement of the appropriate hardware components. An initial prototype baseplate was designed to securely mount the motors, allowing for early-stage testing. A 3S LiPo battery was used as the primary power source for this prototype, shown in Figure 24, with the 5V output from the motor driver used to power the ESP32 microcontroller. This setup enabled successful movement of the robot, validating the fundamental hardware integration.

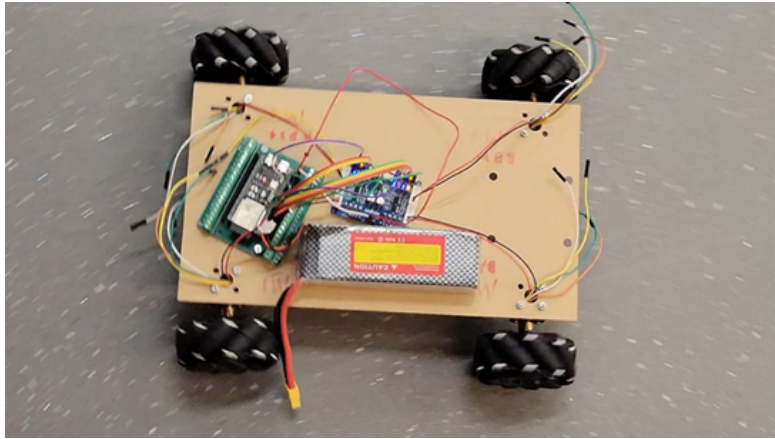


Figure 24: First prototype moving

The second prototype introduced several structural improvements, including dedicated cable routing slots and provisions for securing the battery using zip ties, shown in Figure 25. Mounting holes were added to accommodate the motor driver board, Raspberry Pi, ESP32, and the voltage converter board. The 3S LiPo battery from the initial prototype was replaced with a 4S LiPo, as the former experienced over-discharge and subsequent failure. Additionally, a separate battery bank was integrated to power the Raspberry Pi, addressing the issue of insufficient output from the motor driver's onboard 5V regulator. This iteration successfully achieved stable robot movement and reliable encoder feedback, marking a significant step forward in system integration.

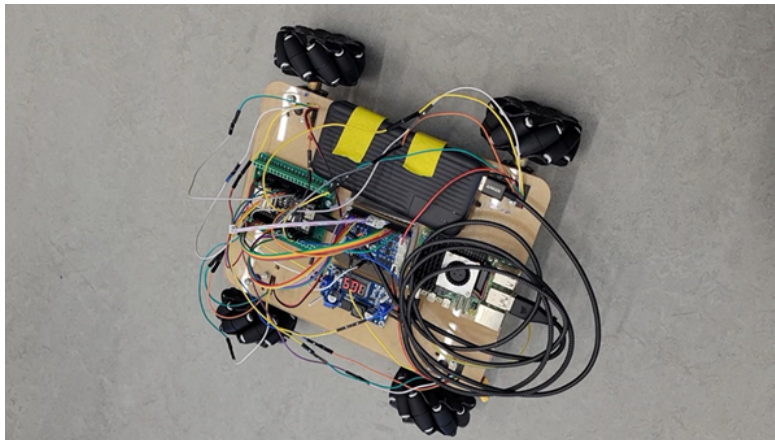


Figure 25: Second prototype

The subsequent prototype introduced additional structural enhancements, including mounting points for a top plate, 3D-printed spacers for elevation, and the top plate itself to accommodate upper-level components, shown in Figures 26 and 28. In the following iteration, the previously used power bank was replaced with a dedicated converter board, and a custom 3D printed ESP32 holder was added to streamline wiring and improve connection stability, shown in Figure 27.

To ensure secure and organized power distribution, a custom wiring harness was developed. This included connectors for the battery, driver board, USB power supply for the camera, and GPIO connectors for powering the Raspberry Pi. During this process, several converter boards were evaluated. The LM2592 module (provided by the university) failed to supply sufficient power, while the LM2956 functioned temporarily but overheated and failed. The XL4016 also proved unreliable due to an unspecified fault. Ultimately, the XL4015E1 converter board from Drok was selected. It met all performance requirements, reliably powering the Raspberry Pi, ESP32, servos, and camera. Moreover, it featured an integrated display for monitoring input voltage, helping to prevent over discharge of the LiPo battery which was a failure encountered in earlier versions.

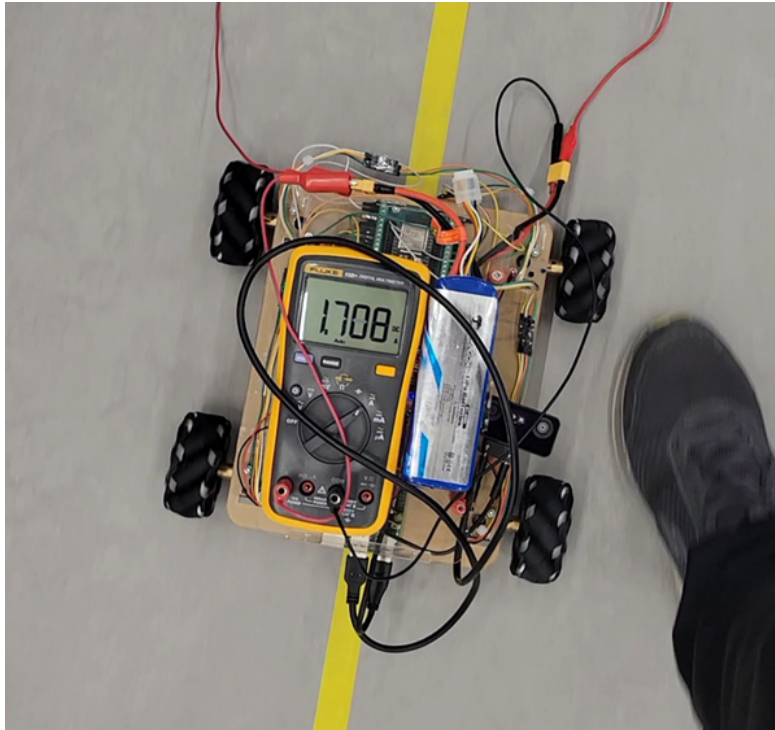


Figure 26: Third prototype moving with current readings for power calculations

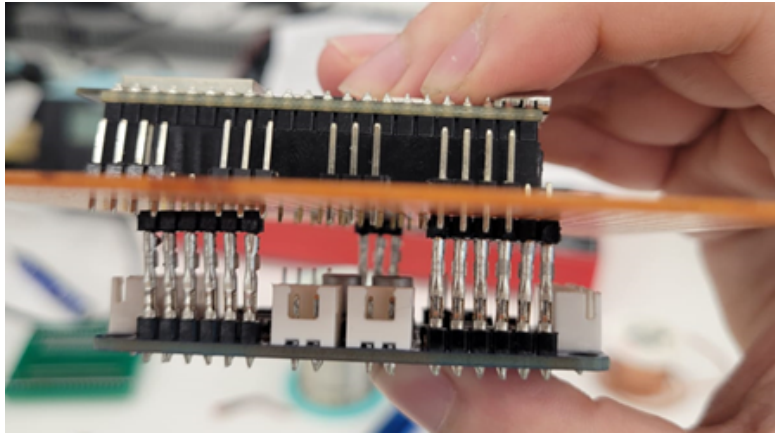


Figure 27: Side profile of custom ESP mount

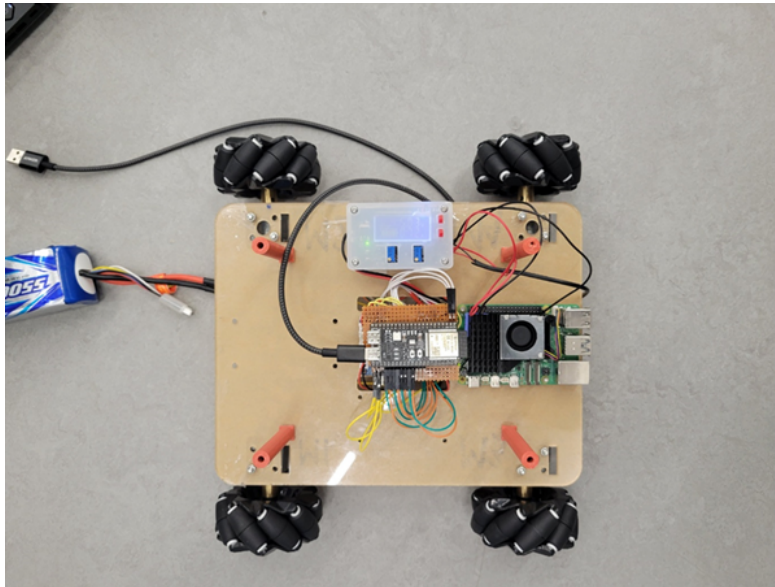


Figure 28: Third prototype with custom ESP mount

The finalized components were 3d printed and assembled to complete the last iteration. Heat-set inserts were used instead of nuts for a cleaner and more finished product. However, during this stage our motor driver began to fail, causing us to remove the esp32 holder to allow for troubleshooting. The original driver board was replaced with two lower quality alternatives, but this setup did not perform as intended. In particular, the robot's ability to strafe

left and right became highly unpredictable and unreliable. Figures 29, 30, and 31 show the fully assembled robot.

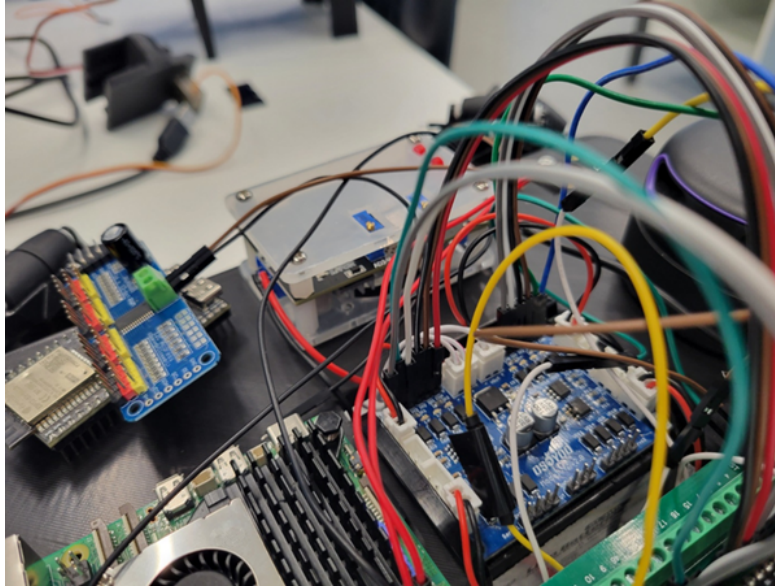


Figure 29: Final wiring underneath top plate

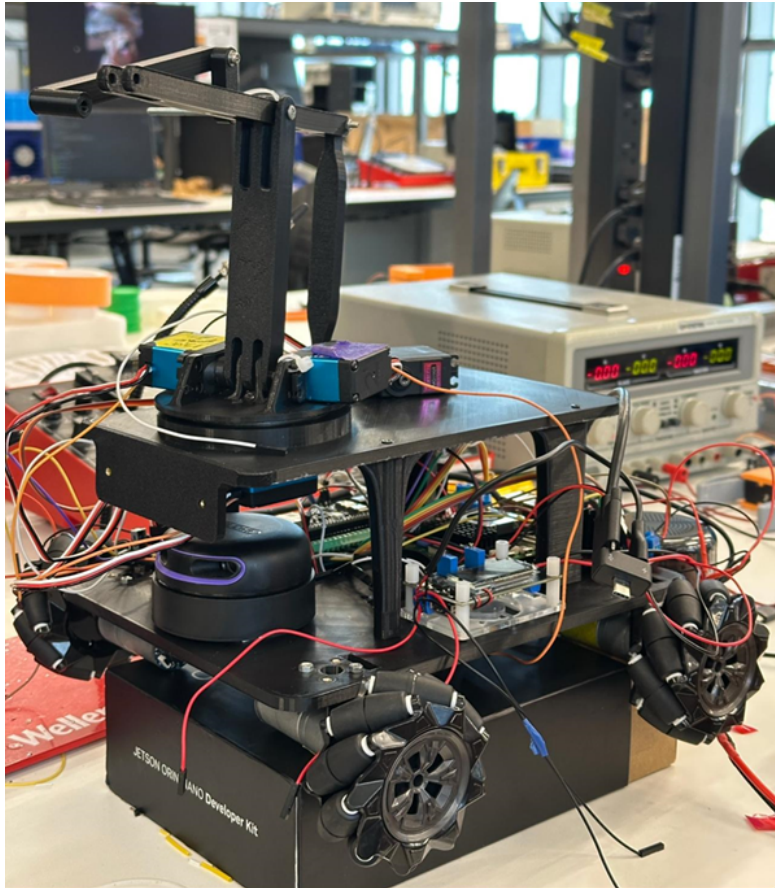


Figure 30: Front view of assembled body

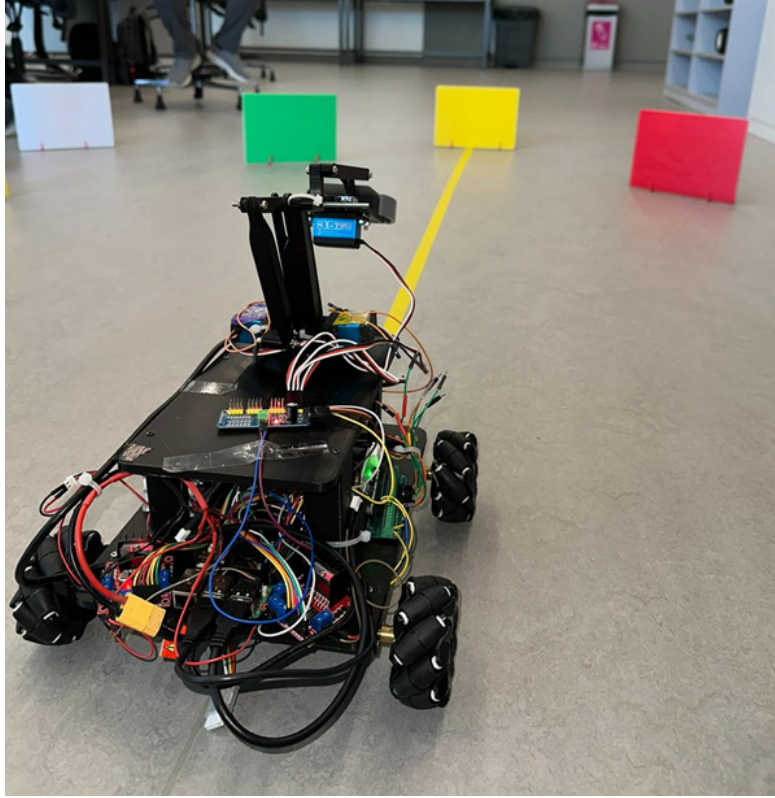


Figure 31: Rear view of assembled body

Development of the robotic arm was carried out in parallel with the design of the top plate. The initial arm prototype was based on a fully defined parametric sketch created in Fusion 360, incorporating joints to simulate and rapidly test variations in the offset 4-bar linkage geometry, as seen in Figures 32 and 33. This design was then laser-cut from acrylic for physical prototyping, visible in Figure 34. The first iteration included only the initial four-bar linkage, which served as a proof of concept to evaluate its mechanical compatibility with the robot chassis. It also tested the feasibility of using the laser cutter to emboss the servo gear profile directly into the acrylic which aimed at reducing both design complexity and overall component size.

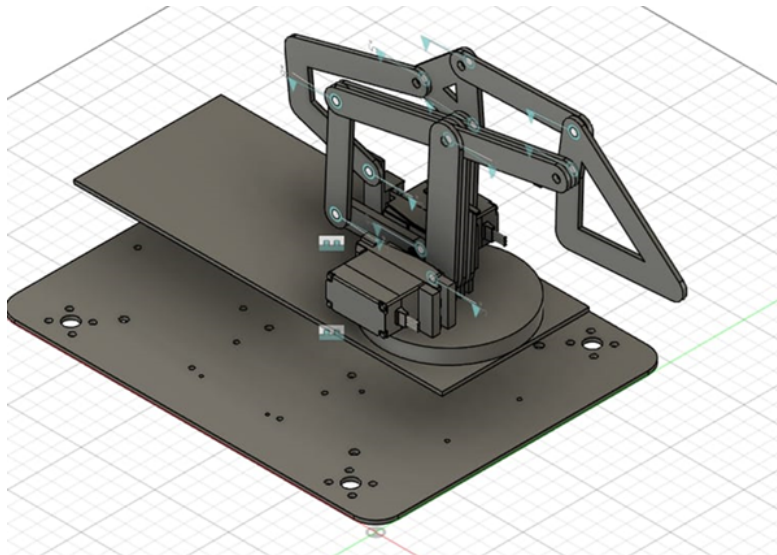


Figure 32: Fusion model of the initial arm

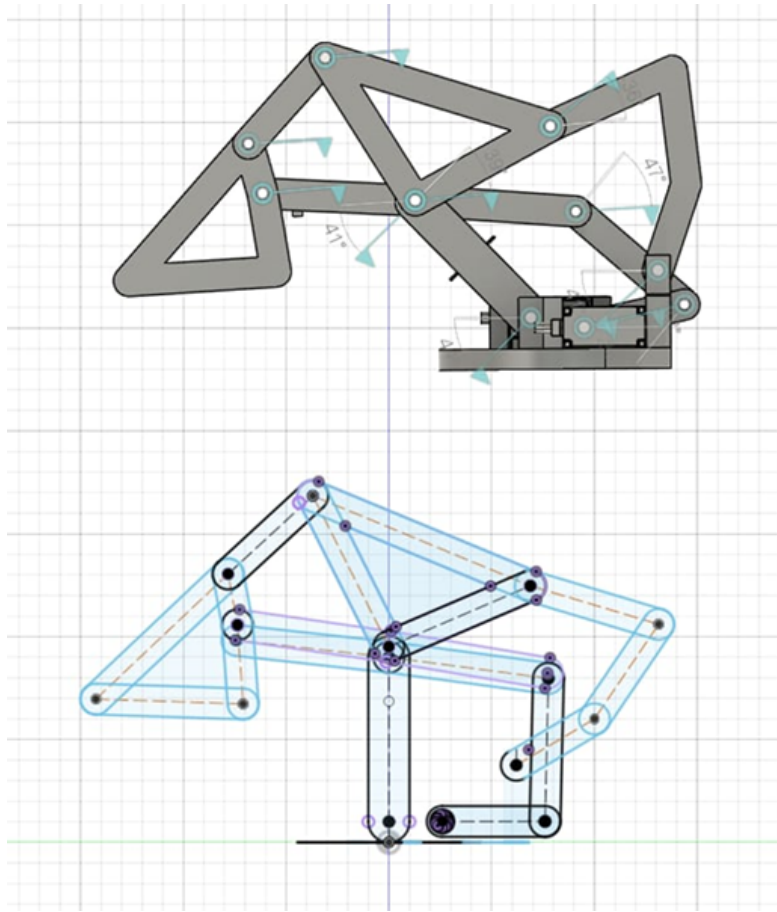


Figure 33: Side profile of the design

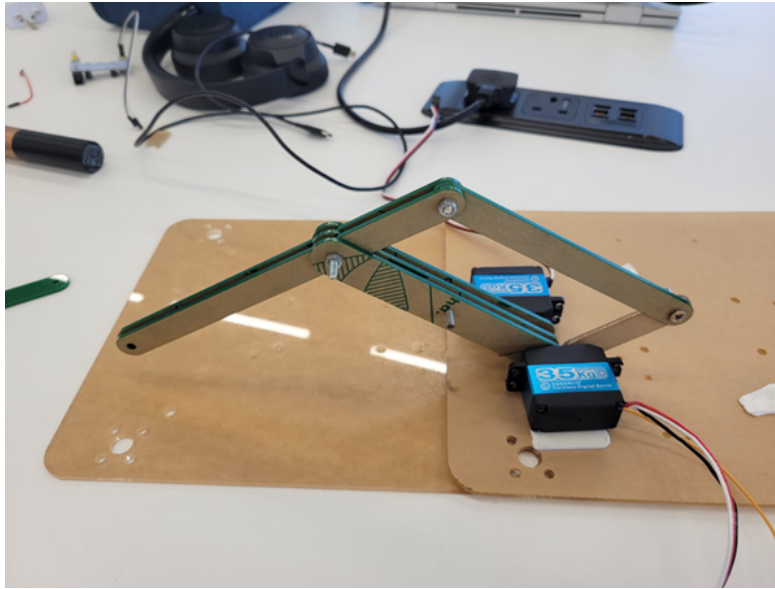


Figure 34: First prototype of the arm

The second prototype, shown in Figure 35, featured an extended reach and introduced a dual 4-bar linkage upright mechanism, enhancing the arm's range of motion and stability. The base of this structure was also laser cut from acrylic to maintain precision and consistency with earlier iterations. Additionally, the top plate was updated to include mounting points for the camera, supporting further integration of perception systems.

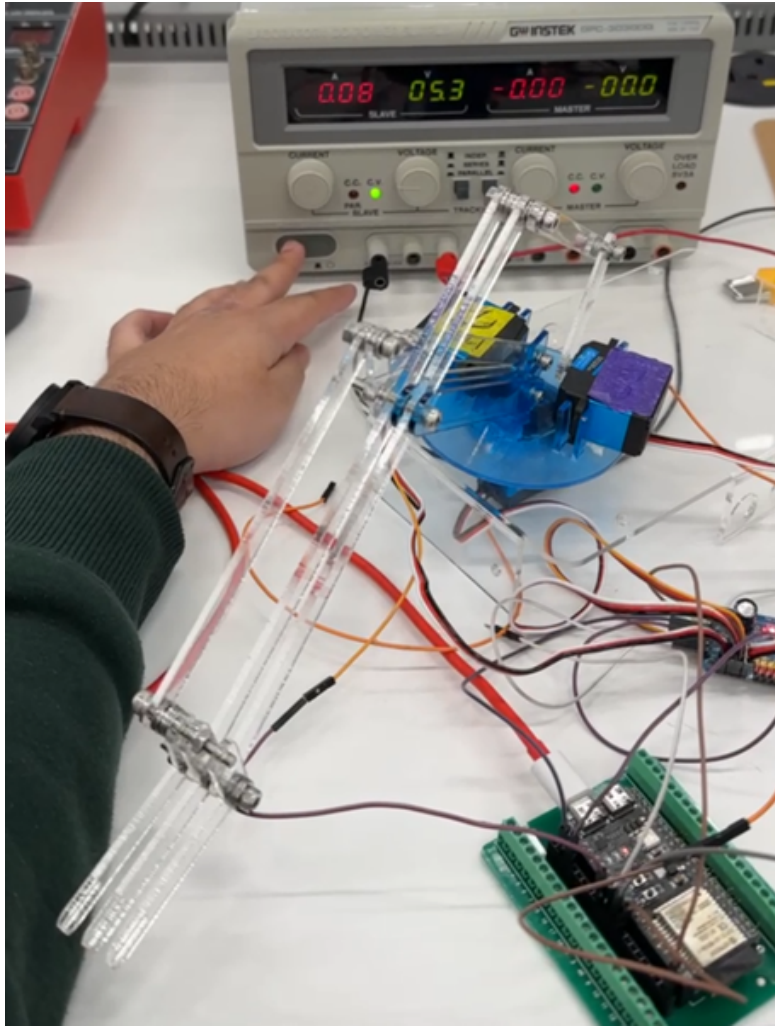


Figure 35: Second prototype of the arm with top plate

The finalized arm design was fully integrated with the top plate and 3D printed for structural strength and precision, both of which are shown in Figures 36 and 37. The top plate itself included a built-in camera holder and standoffs to support the upper components. The base of the arm was embedded directly into the top plate, featuring a custom groove designed to accommodate a slip ring, which helped minimize friction during rotation and improved overall mechanical efficiency.



Figure 36: Final arm and top plate

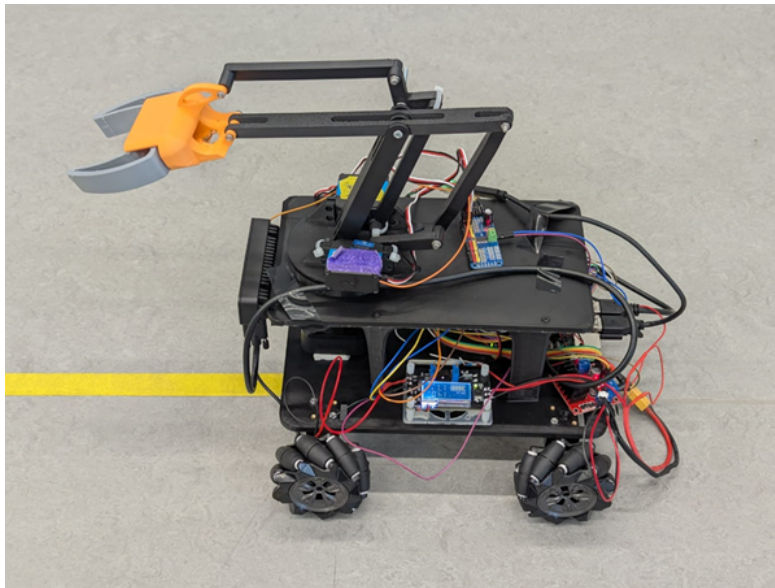


Figure 37: Fully assembled body with arm

The end effector was designed using Fusion 360 and fabricated using TPU through 3D printing. The initial prototype incorporated a mini 9g servo motor

which proved to be insufficient in torque for the application. As a result, a revised version was produced to accommodate a standard-sized servo, providing the necessary strength, torque and reliability. Figures 38 and 39 illustrate the design iterations of the end-effector, while Figure ?? showcases the final gripper in operation.

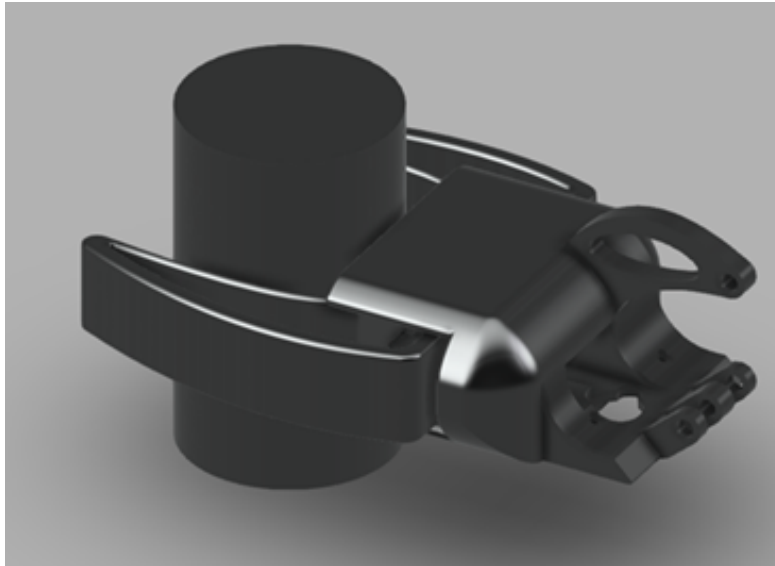


Figure 38: End effector relative to target object

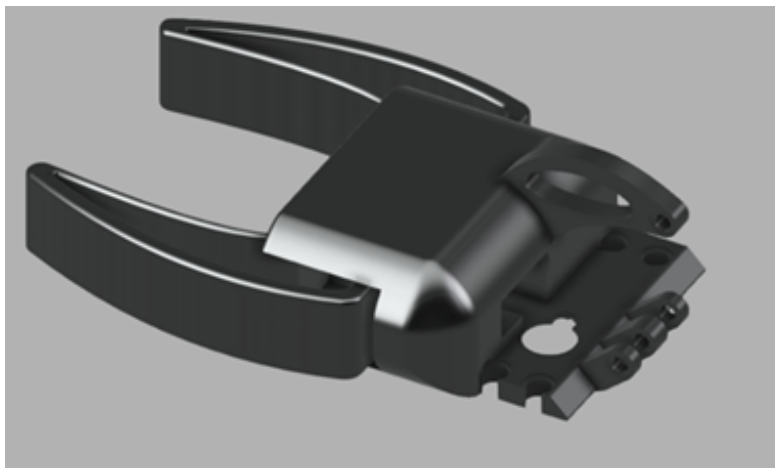


Figure 39: End effector design

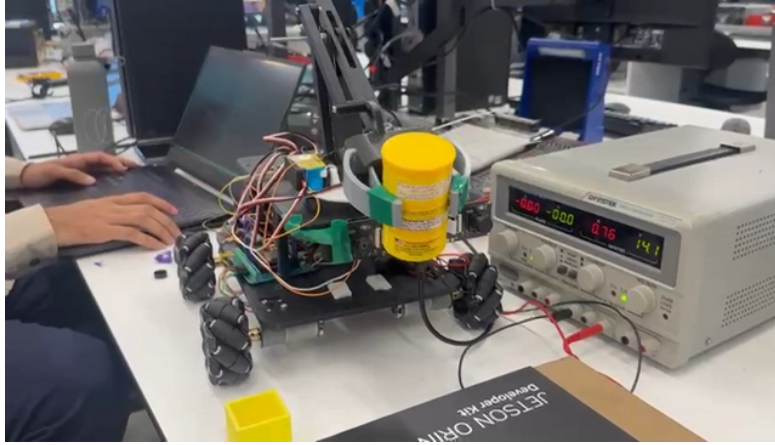


Figure 40: Final gripper grasping the object

4 Results

4.1 Computer simulation

We developed a full robot simulation in the Unity game engine with target points and Aruco markers in the scene to test and iterate on our navigation software in the initial stages of the project. The simulation was capable of simulating both RGB and depth frames for our stereo cameras, a LiDAR, an IMU, as wheels that can be individually commanded with feedback from encoders. The simulation environment can be seen in Figure 41.

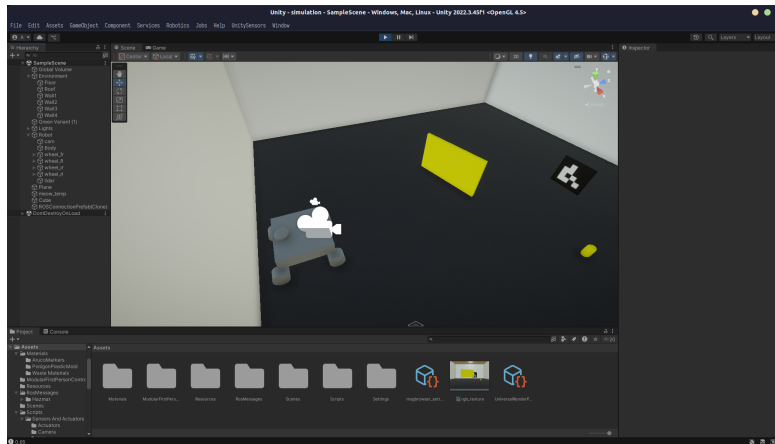


Figure 41: Simulation environment inside the Unity editor

4.2 Testing results

Testing our SLAM algorithm in real life, we were able to map and localise in it later with high accuracy, the produced map can be seen in Figure 42. The odometry node was able to localise direction quite accurately too, with noticeable drift beyond 2 metres of movement.

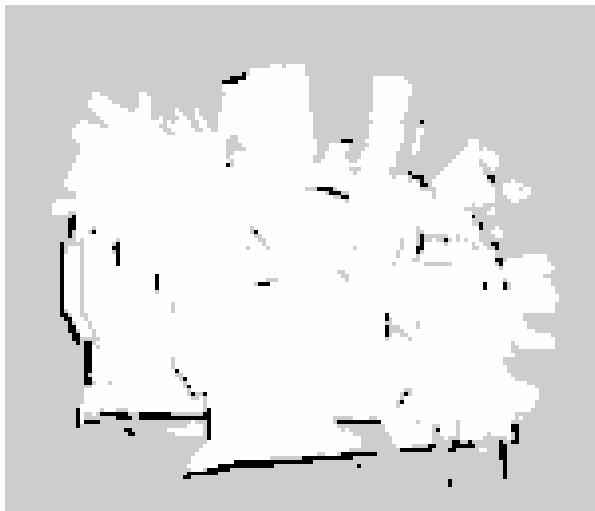


Figure 42: 2D occupancy map of the East end of the EE lab

Our autonomous navigation logic allowed the robot to reach its required target positions with a tolerance of ± 3 cm due to delay in getting a LiDAR reading to actually commanding the robot. For the manipulator, we required manual placement of objects away from the robot, as our logic mainly only changed the rotation of the first joint to track the object.

5 Management and Marketing

5.1 Project Management

The development of Hazmat followed a structured project management approach to ensure timely and effective execution. The project was divided into distinct phases: planning, design, implementation, integration, testing, and final demonstration. Task assignments were made based on individual team members' technical strengths, with regular meetings conducted to monitor progress and resolve arising issues.

A Gantt chart was created using Microsoft Project to plan and track activities throughout the academic semester. This facilitated effective time management, accountability, and coordination within the team. Weekly meetings provided

opportunities for iterative testing and refinement, particularly during critical stages such as sensor integration, SLAM implementation, and mechanical assembly.

Key milestones of the project included the successful integration of the LiDAR and camera systems, enabling real-time perception of the environment. The team created a custom indoor layout to simulate a hazardous waste handling scenario and designed a complete mechanical structure of the robot, which was then 3D printed and assembled. A teleoperation mode was implemented for manual control and testing, followed by the deployment of autonomous navigation using SLAM (Simultaneous Localization and Mapping) combined with LiDAR for dynamic map generation. Object detection and colour-based sorting were successfully demonstrated, allowing the robot to identify, pick, and classify hazardous waste materials based on visual input.

Risk mitigation strategies were implemented to address potential delays in hardware procurement, sensor calibration challenges, and software integration issues.

5.2 Project Finance

Bill of Materials (BOM)

The following table summarizes the major components used in the Hazmat project along with their quantity and approximate cost:

Component	Quantity	Unit Price (AED)	Total Price (AED)
Raspberry Pi 5	1	290	290
ESP32-S3	1	25	25
LiDAR	1	400	400
Stereo Camera	1	1800	1800
DC Motors with Encoders	4	60	240
Servo Motors	4	40	160
Motor Drivers	2	80	160
Mecanum Wheels	4	50	200
3D Printing Material (PLA & TPU)	-	20	100
Misc. Components (buck converter, I2C board, bearings, end-effector)	-	150	150
Total Estimated Cost	3550 AED		

Human Resources

Man-hours were estimated based on the Gantt chart and roles defined within the project. Assuming an hourly rate of AED 25 (for estimation purposes), the

breakdown given in Figure 43.

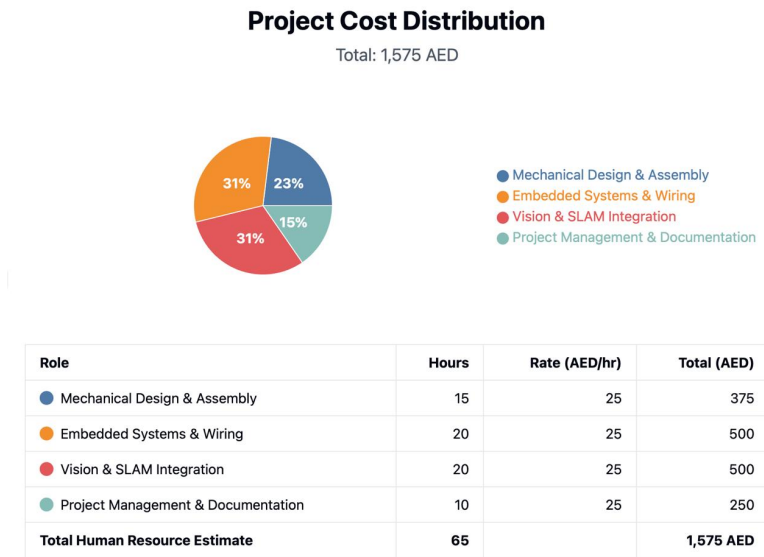


Figure 43: HR cost breakdown

Marketing expenses

To promote and present Hazmat, basic marketing materials were used, including:

- A0-size printed poster: AED 100
- Website cost: AED 1000
- Flyer cost: AED 300

5.3 Marketing

- [Web Design](#)
- [Flyer](#)
- [Poster Presentation](#)

5.4 Risk Management

Strengths	Weaknesses
• Integration of SLAM for autonomous navigation • Accurate object detection and sorting using stereo vision • Custom-built 3 DOF robotic arm with upright end-effector • Mecanum wheel mobility for flexible navigation	• Limited onboard computing power • Dependency on environmental lighting for vision tracking • Limited range of LiDAR used • Project cost limitations affected sensor choices
Opportunities	Threats
• Integration with AI for smarter decision-making • Environmental adaptation for real-world applications • Future scalability to multi-robot systems	• Hardware failure during live demo • Sensor malfunction or misalignment • Competition from commercial-grade autonomous systems

Figure 44: SWOT analysis

5.5 Health, Safety, and Environment

Hazmat was developed with strict consideration for health, safety, and environmental regulations. All mechanical components were securely assembled, and electrical wiring was insulated and properly routed to avoid short circuits or overheating.

To ensure safety during operation:

- The robot speed was limited to prevent accidental collisions.
- Edges on 3D-printed parts were rounded.
- The robotic arm was programmed to operate within a controlled range of motion.
- Voltage regulators were used to maintain stable power delivery.

From an environmental standpoint, the use of 3D-printed PLA - a biodegradable material - was emphasized. The system's purpose itself addresses environmental sustainability by improving hazardous waste handling, thereby reducing risks of human exposure and environmental contamination.

6 Conclusion and Future Work

The Hazmat project represents the outcome of a rigorous engineering effort driven by a shared commitment to innovation, safety, and practical problem-

solving. Tasked with developing a robotic solution for hazardous waste handling, our team designed and implemented an autonomous system aimed at minimizing human exposure in high-risk environments.

Throughout the development process, we applied advanced concepts in robotics and autonomous navigation. Notable technical achievements include the successful implementation of SLAM using Cartographer with LiDAR and IMU data, development of a 3-DOF manipulator with a four-bar linkage for efficient and stable object handling, integration of vision-based object recognition and servoing, and deployment of a ROS-based communication stack using micro-ROS for real-time subsystem control. The system also featured feedback-enabled control for improved precision, an interactive audio feedback module, and a simulation environment built in Unity for iterative testing. These accomplishments demonstrate the feasibility of deploying a fully integrated autonomous mobile manipulator in confined and dynamic industrial settings.

Beyond engineering, the project tested our capabilities in project management, market analysis, sustainability, and ethical engineering. We explored real-world deployment scenarios in sectors such as chemical spill response, industrial site cleanup, and post-disaster operations contexts where automation can significantly improve safety and efficiency.

Despite encountering challenges such as power management issues, hardware failure, and mapping drift, we maintained a solution-oriented approach. Each setback contributed to a cycle of learning and improvement that shaped the final system into a cohesive, functioning prototype capable of navigating environments, detecting waste, and performing autonomous collection and disposal.

In conclusion, Hazmat is more than just a demonstration of robotic capability—it reflects how thoughtful design, interdisciplinary teamwork, and technological innovation can come together to address real-world problems. This project lays a strong foundation for future advancements in autonomous hazardous waste management and presents numerous avenues for future work, including the integration of autonomous charging capabilities, enhancement of visual recognition with machine learning, and expansion into outdoor or GPS-based navigation. Further research and refinement will bring this concept closer to industrial deployment and contribute to a safer, more sustainable future.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, eabm6074, 2022. DOI: [10.1126/scirobotics.abm6074](https://doi.org/10.1126/scirobotics.abm6074). [Online]. Available: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>.

Appendices

A Code repository

All of our code for ROS nodes, ESP32s, and simulations files, as well as the source files for the report, are hosted on a public repository on Github: [link](#).

B Risk assessment matrix

5x5 Risk Assessment Matrix - SLAM Industries Hazmat Robot

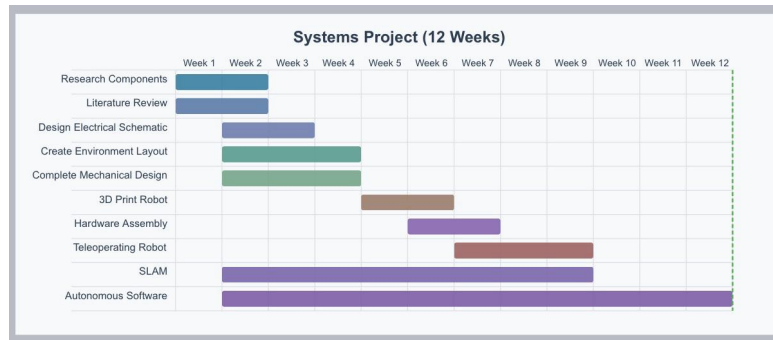
Risk Matrix					
	Insignificant (1)	Minor (2)	Significant (3)	Major (4)	Severe (5)
5 Almost Certain	-	-	R1: Hardware failure	R10: Integration issues	-
4 Likely	-	R4: Navigation obstacles	R2: Software bugs	R3: Sensor calibration	-
3 Moderate	-	R5: Mechanical damage	R6: Arm failure	R7: Electrical wiring	R8: Battery failure
2 Unlikely	-	-	-	R9: Power supply issues	-
1 Rare	-	-	-	R11: Safety hazards	-

Risk Identification:

- R1: Hardware component failure (motors, servos)
- R2: Software bugs in navigation system
- R3: Sensor calibration issues
- R4: Environmental obstacles causing navigation failure
- R5: Mechanical damage during transportation
- R6: Mechanical arm failure during waste pickup
- R7: Electrical wiring issues
- R8: Battery failure during operation
- R9: Inadequate power supply for operations

- R10: Integration issues between subsystems
- R11: Health and safety hazards during testing

C Gantt chart



D Meeting minutes

Meeting Minutes

 **Date: 16/1/2025**

 **Time: 1:30 PM – 2:00 PM**

Attendees

Name	Present
Muhammad Abban	☒
Abdul Maajid Aga	☒
Sufyaan Atif	☒
Moiz Saeed	☒
Laith Muhammad	☒
Aisultan Alpysbay	☒

Agenda

- Research available technologies and existing waste management robotics projects.

Action Items

- Come up with ideas and suggestions for robot design.
-

Meeting Minutes

 **Date: 20/1/2025**

 **Time: 1:00 PM – 3:00 PM**

Attendees

Name	Present
Muhammad Abban	☒
Abdul Maajid Aga	☒
Sufyaan Atif	☒
Moiz Saeed	☒
Laith Muhammad	☒
Aisultan Alpysbay	☒

Agenda

- Discuss possible robot designs and available projects.
- Decide on an effective gripper for picking and placing hazardous objects.

Notes

- Forklift with a gripper(bimanual) with 2 dof; if we have trays

- Web UI, external processor on computer
- Visual SLAM (available packages such as ORBSLAM)
- Use mecanum(omni-directional) wheels on our robot

Action Items

- Robot design (sketch and CAD)
-

Meeting Minutes

 **Date: 21/1/2025**

 Time: 3:20 PM – 4:00 PM

Attendees

Name	Present
Muhammad Abban	☒
Abdul Maajid Aga	☒
Sufyaan Atif	☒
Moiz Saeed	☒
Laith Muhammad	☒
Aisultan Alpysbay	☒

Agenda

- Discuss workflow of our project
- Distribute tasks to each team member

Notes

- Starting with setting up the environment

Action Items

- CAD design for our robot
 - Components list and cost
-

Meeting Minutes

 **Date: 23/1/2025**

 Time: 1:30 PM – 4:00 PM

Attendees

Name	Present
Muhammad Abban	☒
Abdul Maajid Aga	☒
Sufyaan Atif	☒
Moiz Saeed	☒
Laith Muhammad	☒
Aisultan Alpysbay	☒

Agenda

- Discuss the environment with team leaders
- Assign tasks to team members

Notes

- Environment will have same start and stop points
- The layout of the environmental will be consistent
- Team Leaders will research existing environments and finalize a layout

Action Items

- Prepare complete mechanical design
- Animate the picking and dropping mechanism

Meeting Minutes

 **Date: 30/1/2025**

 **Time: 1:30 PM – 4:00 PM**

Attendees

Name	Present
Muhammad Abban	☒
Abdul Maajid Aga	☒
Sufyaan Atif	☒
Moiz Saeed	☒
Laith Muhammad	☒
Aisultan Alpysbay	☒

Agenda

- Finalize mechanical CAD design
- Test electrical components

Action Items

- Research arm mechanisms

Meeting Minutes

 **Date: 4/2/2025**

 Time: 3:30 PM – 4:30 PM

Attendees

Name	Present
Muhammad Abban	☒
Abdul Maajid Aga	☒
Sufyaan Atif	☒
Moiz Saeed	☒
Laith Muhammad	☒
Aisultan Alpysbay	☒

Agenda

- Everyone sketches a design for the robot arm

Notes

- Pugh matrix used to select the best design

Action Items

- Update design notebook
-

Meeting Minutes

 **Date: 6/2/2025**

 Time: 2:30 PM – 4:30 PM

Attendees

Name	Present
Muhammad Abban	☒
Abdul Maajid Aga	☒
Sufyaan Atif	☒
Moiz Saeed	☒
Laith Muhammad	☒
Aisultan Alpysbay	☒

Agenda

- Discuss workflow of our project
- Assign new tasks to members

Notes

No notes recorded for this meeting

Action Items

- Individual tasks assigned to members
-

Meeting Minutes

 Date: 13/2/2025

 Time: 2:00 PM – 4:30 PM

Attendees

Name	Present
Muhammad Abban	☒
Abdul Maajid Aga	☒
Sufyaan Atif	☒
Moiz Saeed	☒
Laith Muhammad	☒
Aisultan Alpysbay	☒

Agenda

- Demonstrate robot wheels running on command using ROS nodes

Action Items

- Individual tasks assigned to members
-

Meeting Minutes

 Date: 20/2/2025

 Time: 3:00 PM – 4:30 PM

Attendees

Name	Present
Muhammad Abban	☒
Abdul Maajid Aga	☒
Sufyaan Atif	☒
Moiz Saeed	☒
Laith Muhammad	☒
Aisultan Alpysbay	☒

Agenda

- Control wheels and print RPM

Action Items

- Teleoperation of robot

Meeting Minutes

 **Date: 28/2/2025**

 Time: 1:00 PM – 2:30 PM

Attendees

Name	Present
Muhammad Abban	☒
Abdul Maajid Aga	☒
Sufyaan Atif	☒
Moiz Saeed	☒
Laith Muhammad	☒
Aisultan Alpysbay	☒

Agenda

- Implement SLAM for mapping

Action Items

- Assemble robot arm and work on IK

Meeting Minutes

 **Date: 5/3/2025**

🕒 Time: 2:00 PM – 4:30 PM

Attendees

Name	Present
Muhammad Abban	☒
Abdul Maajid Aga	☒
Sufyaan Atif	☒
Moiz Saeed	☒
Laith Muhammad	☒
Aisultan Alpysbay	☒

Agenda

- Test pick and place mechanism of arm

Action Items

- Color detection using Lidar